Chapter 10 ▪ Concurrency

You are probably familiar with *multitasking*—your operating system's ability to have more than one program working at what seems like the same time. For example, you can print while editing or downloading your email. Nowadays, you are likely to have a computer with more than one CPU, but the number of concurrently executing processes is not limited by the number of CPUs. The operating system assigns CPU time slices to each process, giving the impression of parallel activity.

Multithreaded programs extend the idea of multitasking by taking it one level lower: Individual programs will appear to do multiple tasks at the same time. Each task is executed in a *thread*, which is short for thread of control. Programs that can run more than one thread at once are said to be *multithreaded*.

So, what is the difference between multiple *processes* and multiple *threads*? The essential difference is that while each process has a complete set of its own variables, threads share the same data. This sounds somewhat risky, and indeed it can be, as you will see later in this chapter. However, shared variables make communication between threads more efficient and easier to program than interprocess communication. Moreover, on some operating systems, threads are more "lightweight" than processes—it takes less overhead to create and destroy individual threads than it does to launch new processes.

Multithreading is extremely useful in practice. For example, a browser should be able to simultaneously download multiple images. A web server needs to be able to serve concurrent requests. Graphical user interface (GUI) programs have a separate thread for gathering user-interface events from the host operating environment. This chapter shows you how to add multithreading capability to your Java applications.

Fair warning: Concurrent programming can get very complex. In this chapter, I cover all the tools that an application programmer is likely to need. However, for more intricate system-level programming, I suggest that you turn to a more advanced reference, such as *Java Concurrency in Practice* by Brian Goetz et al. (Addison-Wesley Professional, 2006).

10.1. Running Threads

Here is a simple procedure for running a task in a separate thread:

1. Place the code for the task into the `run` method of a class that implements the `Runnable` interface. That interface is very simple, with a single method:

```
public interface Runnable
{
  void run();
}
```

   Since `Runnable` is a functional interface, you can make an instance with a lambda expression:

```
Runnable r = () ->
  {
    task code
  };
```

2. Construct a `Thread` object from the `Runnable`:

```
var t = new Thread(r);
```

3. Start the thread:

```
t.start();
```

Let us look at a simple program that uses threads to move money between bank accounts. We make use of a `Bank` class that stores the balances of a given number of accounts. The `transfer` method transfers an amount from one account to another. See **Listing 10.2** for the implementation.

In the first thread, we will move money from account 0 to account 1:

```
Runnable r = () ->
  {
    try
    {
      for (int i = 0; i < STEPS; i++)
      {
        double amount = MAX_AMOUNT * Math.random();
```

```
            bank.transfer(0, 1, amount);
            Thread.sleep((int) (DELAY * Math.random()));
          }
        }
        catch (InterruptedException e)
        {
        }
      };
    var t = new Thread(r);
    t.start();
```

For a given number of steps, this thread transfers a random amount, and then sleeps for a random delay.

We need to catch an `InterruptedException` that the `sleep` method threatens to throw. We will discuss this exception in **Section 10.3.2**. Typically, interruption is used to request that a thread terminates. Accordingly, our `run` method exits when an `InterruptedException` occurs.



Note: Calling `Math.random()` from a large number of different threads is slightly inefficient. For this and the following demonstration programs, we do not care. But see **Section 10.5.14** for a more performant approach.

Our program starts a second thread as well that moves money from account 2 to account 3. When you run this program, you get a printout like this:

```
Thread[Thread-1,5,main]    606.77 from 2 to 3 Total Balance:  400000.00
Thread[Thread-0,5,main]     98.99 from 0 to 1 Total Balance:  400000.00
Thread[Thread-1,5,main]    476.78 from 2 to 3 Total Balance:  400000.00
Thread[Thread-0,5,main]    653.64 from 0 to 1 Total Balance:  400000.00
Thread[Thread-1,5,main]    807.14 from 2 to 3 Total Balance:  400000.00
Thread[Thread-0,5,main]    481.49 from 0 to 1 Total Balance:  400000.00
Thread[Thread-0,5,main]    203.73 from 0 to 1 Total Balance:  400000.00
Thread[Thread-1,5,main]    111.76 from 2 to 3 Total Balance:  400000.00
Thread[Thread-1,5,main]    794.88 from 2 to 3 Total Balance:  400000.00
. . .
```

As you can see, the output of the two threads is interleaved, showing that they run concurrently. In fact, sometimes the output is a little messier when two output lines are interleaved.

That's all there is to it! You now know how to run tasks concurrently. The remainder of this chapter tells you how to control the interaction between threads.

The complete code is shown in **Listing 10.1**.



Caution: Do *not* call the `run` method of the `Thread` class or the `Runnable` object. Calling the `run` method directly merely executes the task in the *same* thread—no new thread is started. Instead, call the `Thread.start` method. It creates a new thread that executes the `run` method.



Note: You can also define a thread by forming a subclass of the `Thread` class, like this:

```
class MyThread extends Thread
{
  public void run()
  {
    task code
  }
}
```

Then you construct an object of the subclass and call its `start` method. However, this approach is no longer recommended. You should decouple the *task* that is to be run in parallel from the *mechanism* of running it. As you will see throughout this chapter, there are many ways of scheduling the execution of a task.

Listing 10.1 threads/ThreadTest.java

```
1  package threads;
2
3  /**
4   * @version 1.30 2004-08-01
5   * @author Cay Horstmann
6   */
7  public class ThreadTest
8  {
9    public static final int DELAY = 10;
10   public static final int STEPS = 100;
11   public static final double MAX_AMOUNT = 1000;
12
13   public static void main(String[] args)
```

```
14    {
15       var bank = new Bank(4, 100000);
16       Runnable task1 = () ->
17          {
18             try
19             {
20                for (int i = 0; i < STEPS; i++)
21                {
22                   double amount = MAX_AMOUNT * Math.random();
23                   bank.transfer(0, 1, amount);
24                   Thread.sleep((int) (DELAY * Math.random()));
25                }
26             }
27             catch (InterruptedException e)
28             {
29             }
30          };
31
32       Runnable task2 = () ->
33          {
34             try
35             {
36                for (int i = 0; i < STEPS; i++)
37                {
38                   double amount = MAX_AMOUNT * Math.random();
39                   bank.transfer(2, 3, amount);
40                   Thread.sleep((int) (DELAY * Math.random()));
41                }
42             }
43             catch (InterruptedException e)
44             {
45             }
46          };
47
48       new Thread(task1).start();
49       new Thread(task2).start();
50    }
51 }
```

Listing 10.2 threads/Bank.java

```
1  package threads;
2
3  import java.util.*;
```

```
4
5  /**
6   * A bank with a number of bank accounts.
7   */
8  public class Bank
9  {
10   private final double[] accounts;
11
12   /**
13    * Constructs the bank.
14    * @param n the number of accounts
15    * @param initialBalance the initial balance for each account
16    */
17   public Bank(int n, double initialBalance)
18   {
19     accounts = new double[n];
20     Arrays.fill(accounts, initialBalance);
21   }
22
23   /**
24    * Transfers money from one account to another.
25    * @param from the account to transfer from
26    * @param to the account to transfer to
27    * @param amount the amount to transfer
28    */
29   public void transfer(int from, int to, double amount)
30   {
31     if (accounts[from] < amount) return;
32     System.out.print(Thread.currentThread());
33     accounts[from] -= amount;
34     System.out.printf(" %10.2f from %d to %d", amount, from, to);
35     accounts[to] += amount;
36     System.out.printf(" Total Balance: %10.2f%n", getTotalBalance());
37   }
38
39   /**
40    * Gets the sum of all account balances.
41    * @return the total balance
42    */
43   public double getTotalBalance()
44   {
45     double sum = 0;
46
47     for (double a : accounts)
48       sum += a;
```

```
49
50        return sum;
51     }
52
53     /**
54      * Gets the number of accounts in the bank.
55      * @return the number of accounts
56      */
57     public int size()
58     {
59        return accounts.length;
60     }
61  }
```

java.lang.Thread 1.0

- `Thread(Runnable target)`
  constructs a new thread that calls the `run()` method of the specified target.
- `void start()`
  starts this thread, causing the `run()` method to be called. This method will return immediately. The new thread runs concurrently.
- `void run()`
  calls the `run` method of the associated `Runnable`.
- `static void sleep(long millis)`
- `static void sleep(Duration duration)` 19
  sleeps for the given number of milliseconds.

java.lang.Runnable 1.0

- `void run()`
  must be overridden and supplied with instructions for the task that you want to have executed.

10.2. Thread States

Threads can be in one of six states:

- New
- Runnable
- Blocked

- Waiting
- Timed waiting
- Terminated

Each of these states is explained in the sections that follow.

To determine the current state of a thread, simply call the `getState` method.

10.2.1. New Threads

When you create a thread with the `new` operator—for example, `new Thread(r)` —the thread is not yet running. This means that it is in the *new* state. When a thread is in the new state, the program has not started executing code inside of it. A certain amount of bookkeeping needs to be done before a thread can run.

10.2.2. Runnable Threads

Once you invoke the `start` method, the thread is in the *runnable* state. A runnable thread may or may not actually be running. It is up to the thread scheduler to give the thread time to run. (The Java specification does not call this a separate state, though. A running thread is still in the runnable state.)

Once a thread is running, it doesn't necessarily keep running. In fact, it is desirable that running threads occasionally pause so that other threads have a chance to run. The details of thread scheduling depend on the services that the operating system provides, and on the nature of the thread.

As of Java 21, there are two kinds of threads: *platform threads* and *virtual threads*. A Java platform thread corresponds to a thread provided by the operating system, which schedules its execution. Virtual threads run on platform threads and are scheduled by the Java runtime. As you will see in **Section 10.3.1**, virtual threads are used in applications with many more concurrent tasks than available platform threads.

Each processor can run one platform thread at a time. On a machine with multiple processors, you can have multiple threads run in parallel. If there are more runnable threads than available processors, some of them are not actually running. They need to be scheduled for execution.

Platform threads use *preemptive scheduling*. Each running thread is given a slice of time to perform its task. When that slice of time is exhausted, the operating system *preempts* the

thread and gives another thread an opportunity to work. When selecting the next platform thread, the operating system takes into account the thread *priorities*—see **Section 10.3.6** for more information.

Virtual threads use *cooperative scheduling*. A virtual thread loses control only when it calls the `yield` method, or when it is blocked.

`java.lang.Thread` **1.0**

- `static void yield()`
  signals to the scheduler that this thread is willing to yield execution to another thread. Note that this is a static method.

10.2.3. Blocked and Waiting Threads

When a thread is blocked or waiting, it is temporarily inactive. It doesn't execute any code and consumes minimal resources. It is up to the thread scheduler to reactivate it. The details depend on how the inactive state was reached.

- When the thread tries to acquire an intrinsic object lock (but not a `Lock` in the `java.util.concurrent` library) that is currently held by another thread, it becomes *blocked*. (We discuss `java.util.concurrent` locks in **Section 10.5.3** and intrinsic object locks in **Section 10.5.6**.) The thread becomes unblocked when all other threads have relinquished the lock and the thread scheduler has allowed this thread to hold it.
- When the thread waits for another thread to notify the scheduler of a condition, it enters the *waiting* state. We discuss conditions in **Section 10.5.4**. This happens by calling the `Object.wait` or `Thread.join` methods, or by waiting for a `Lock` or `Condition` in the `java.util.concurrent` library.
- Several methods have a timeout parameter. Calling them causes the thread to enter the *timed waiting* state. This state persists either until the timeout expires or the appropriate notification has been received. Methods with timeout include `Thread.sleep` and the timed versions of `Object.wait`, `Thread.join`, `Lock.tryLock`, and `Condition.await`.

Figure 10.1 shows the states that a thread can have and the possible transitions from one state to another.

In practice, the difference between the blocked and waiting states is not usually significant. We will often say that a thread is blocked when it is in the blocked, waiting, or timed

waiting state. When a thread is blocked or waiting (or, of course, when it terminates), another thread will be scheduled to run.

When a thread is reactivated (for example, because its timeout has expired or it has succeeded in acquiring a lock), it becomes runnable and is eligible for being scheduled.

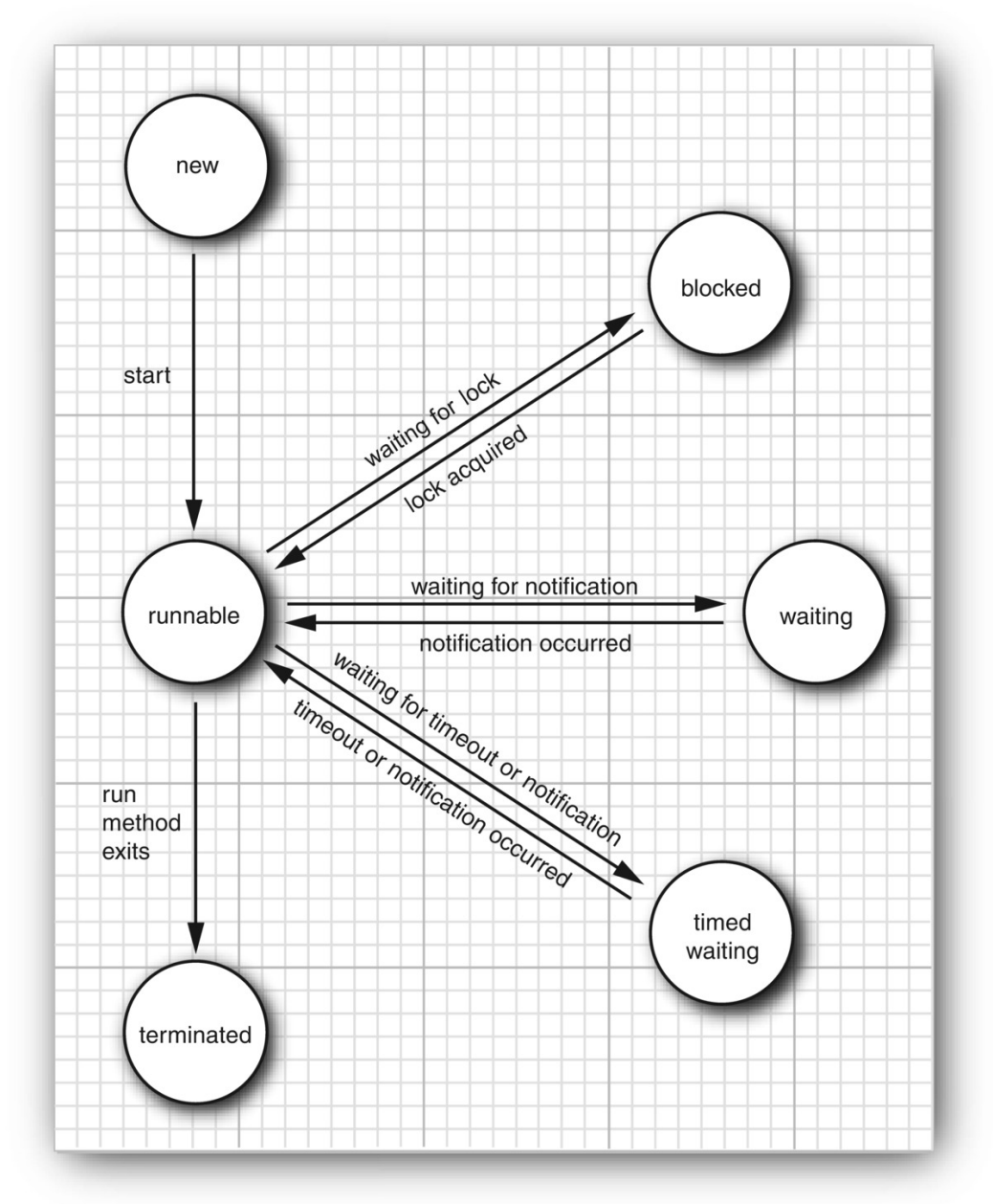


Figure 10.1: Thread states

10.2.4. Terminated Threads

A thread is terminated for one of two reasons:

- It dies a natural death because the `run` method exits normally.
- It dies abruptly because an uncaught exception terminates the `run` method.

java.lang.Thread 1.0

- void join()
  waits for the specified thread to terminate.
- void join(long millis)
- void join(Duration duration) 19
  wait for the specified thread to terminate or for the specified number of milliseconds or duration to pass.
- Thread.State getState() 5.0
  gets the state of this thread: one of NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, or TERMINATED.
- void stop()
- void suspend()
- void resume()
  These methods are deprecated for removal and, as of Java 21, throw an UnsupportedOperationException.

10.3. Thread Properties

In the following sections, we discuss the various properties of threads: platform and virtual threads, the interrupted status, daemon threads, handlers for uncaught exceptions, as well as some legacy features that you should not use.

10.3.1. Virtual Threads

By default, a Java thread runs on a *platform thread* provided by the operating system in which the Java virtual machine executes. For many workloads, this is a good arrangement since operating systems have evolved to do a good job of thread scheduling.

However, platform threads are not lightweight. They require a few thousand CPU instructions to start, and they consume (or at least reserve) a significant amount of memory. This limits the number of platform threads that the CPU can handle.

This is a problem for some classes of applications. A typical web application would like to serve a very large number of concurrent requests, many more than there are platform threads. The application can handle the workload because the tasks don't do CPU-intensive work. Each task spends most of its time blocking on database queries or other external requests.

One solution is to use a non-blocking API for the requests. Instead of a synchronous call

```
response = service.request(parameters);
process(response);
```

pass the processing code as a callback:

```
service.request(parameters, response -> process(response));
```

Then the call to `request` returns immediately without blocking, freeing its thread to serve additional requests. The callback is invoked when the response is available.

Non-blocking APIs provide high throughput, but at a steep cost for the programmer. Instead of using familiar programming constructs for sequences, branches, loops, and exception handling, all program logic must be placed into callback code.

Virtual threads, available in Java 21, are an attractive alternative. A virtual thread is *mounted* on a platform thread and *unmounted* when it encounters a blocking operation. When the result of the operation becomes available, the virtual thread can again be scheduled, and it is remounted when a platform thread becomes available.

The term "virtual thread" is meant to be analogous to virtual memory. Just like virtual memory is mapped to a smaller amount of actual RAM, virtual threads are mounted on a smaller number of platform threads. These platform threads are called the *carrier threads*. By default, the scheduler uses one carrier thread per processor. This can be tuned with the `jdk.virtualThreadScheduler.parallelism` VM option.

Virtual threads have the same API as platform threads. A simple way to create and start a virtual thread is:

```
Thread t = Thread.startVirtualThread(myRunnable);
```

The `isVirtual` method of the `Thread` class returns `true` when a thread is virtual.

java.lang.Thread 1.0

- boolean isVirtual() 21
  returns true when this thread is virtual.
- static Thread startVirtualThread(Runnable task) 21
  starts and returns a virtual thread running the given task.

10.3.2. Thread Interruption

A thread terminates when its run method returns—by executing a return statement, after executing the last statement in the method body, or if an exception occurs that is not caught in the method.



Caution: In the initial release of Java, there also was a stop method that another thread could call to terminate a thread. However, that method was soon recognized to be unsafe and now throws an UnsupportedOperationException instead of stopping the thread.

There is no way to *force* a thread to terminate. However, the interrupt method can be used to *request* termination of a thread.

When the interrupt method is called on a thread, the *interrupted status* of the thread is set. This is a boolean flag that is present in every thread. Each thread should occasionally check whether it has been interrupted.

To find out whether the interrupted status was set, first call the static Thread.currentThread method to get the current thread, and then call the isInterrupted method:

```
while (!Thread.currentThread().isInterrupted() && more work to do)
{
  do more work
}
```

However, if a thread is blocked, it cannot check the interrupted status. This is where the InterruptedException comes in. When the interrupt method is called on a thread that blocks on a call such as sleep or wait, the blocking call is terminated by an InterruptedException. There are blocking I/O calls that cannot be interrupted. Then the InterruptedException is thrown when the I/O operation has completed.

There is no language requirement that a thread which is interrupted should terminate. Interrupting a thread simply grabs its attention. The interrupted thread can decide how to react to the interruption. Some threads are so important that they should handle the exception and continue. But far more often, a thread will simply want to interpret an interruption as a request for termination. The `run` method of such a thread has the following form:

```
Runnable r = () ->
  {
    try
    {
      . . .
      while (!Thread.currentThread().isInterrupted() && more work to do)
      {
        do more work
      }
    }
    catch (InterruptedException e)
    {
      // thread was interrupted
    }
    finally
    {
      cleanup, if required
    }
    // exiting the run method terminates the thread
  };
```

The `isInterrupted` check is neither necessary nor useful if you call the `sleep` method (or another interruptible method) after every work iteration. If you call the `sleep` method when the interrupted status is set, it doesn't sleep. Instead, it clears the status (!) and throws an `InterruptedException`. Therefore, if your loop calls `sleep`, don't check the interrupted status. Instead, catch the `InterruptedException`, like this:

```
Runnable r = () ->
  {
    try
    {
      . . .
      while (more work to do)
      {
        do more work
```

```
        Thread.sleep(delay);
      }
    }
    catch (InterruptedException e)
    {
      // thread was interrupted during sleep
    }
    finally
    {
      cleanup, if required
    }
    // exiting the run method terminates the thread
  };
```



Note: There are two very similar methods, `interrupted` and `isInterrupted`. The `interrupted` method is a static method that checks whether the *current* thread has been interrupted. Furthermore, calling the `interrupted` method *clears* the interrupted status of the thread. On the other hand, the `isInterrupted` method is an instance method that you can use to check whether any thread has been interrupted. Calling it does not change the interrupted status.

You'll find lots of published code in which the `InterruptedException` is squelched at a subtask level, like this:

```
  void mySubTask()
  {
    . . .
    try
    {
      sleep(delay);
    }
    catch (InterruptedException e)
    {
      // Nothing here--BAD
    }
    . . .
  }
```

Don't do that! If you do, the task cannot be interrupted while it is sleeping. You have two reasonable choices:

- Tag your method with `throws InterruptedException` and drop the `try` block. Then the caller (or, ultimately, the `run` method) can catch it.

```
void mySubTask() throws InterruptedException
{
  . . .
  sleep(delay);
  . . .
}
```

- If you cannot do that, set the interrupted status in the `catch` clause. Then the caller can test it.

```
void mySubTask()
{
  . . .
  try
  {
    sleep(delay);
  }
  catch (InterruptedException e)
  {
    Thread.currentThread().interrupt();
  }
  . . .
}
```

`java.lang.Thread` **1.0**

- `void interrupt()`
  sends an interrupt request to a thread. The interrupted status of the thread is set to `true`. If the thread is currently blocked, then an `InterruptedException` is thrown.
- `static boolean interrupted()`
  tests whether the *current* thread (that is, the thread that is executing this instruction) has been interrupted. Note that this is a static method. The call has a side effect—it resets the interrupted status of the current thread to `false`.
- `boolean isInterrupted()`
  tests whether a thread has been interrupted. Unlike the `static interrupted` method, this call does not change the interrupted status of the thread.

- `static Thread currentThread()`
  returns the `Thread` object representing the currently executing thread.

10.3.3. Daemon Threads

You can turn a thread into a *daemon thread* by calling

```
t.setDaemon(true);
```

There is nothing demonic about such a thread. A daemon is simply a thread that has no other role in life than to serve others. Examples are timer threads that send regular "timer ticks" to other threads or threads that clean up stale cache entries. When only daemon threads remain, the virtual machine exits. There is no point in keeping the program running if all remaining threads are daemons.

All virtual threads are daemon threads. Calling `setDaemon(false)` has no effect on a virtual thread.

`java.lang.Thread` **1.0**

- `void setDaemon(boolean isDaemon)`
  marks this thread as a daemon thread or a user thread. This method must be called before the thread is started.

10.3.4. Thread Names and Ids

By default, threads have catchy names such as `Thread-2`. You can set any name with the `setName` method:

```
var t = new Thread(runnable);
t.setName("Web crawler");
```

That can be useful in thread dumps.

Each thread also has a positive ID number that you can retrieve as

```
long id = t.threadId();
```



Caution: There is also a `getId` method. It is deprecated because it was not defined as `final`. Someone might override it to return something other than the thread ID. Call the final `threadId` method instead.

`java.lang.Thread` **1.0**

- `void setName(String name)`
- `String getName()`
  sets or gets the name of this thread.
- `long threadId()` **19**
  returns the unique ID of this thread.

10.3.5. Handlers for Uncaught Exceptions

The `run` method of a thread cannot throw any checked exceptions, but it can be terminated by an unchecked exception. In that case, the thread terminates.

However, there is no `catch` clause to which the exception can be propagated. Instead, just before the thread dies, the exception is passed to a handler for uncaught exceptions.

The handler must belong to a class that implements the `Thread.UncaughtExceptionHandler` interface. That interface has a single method,

```
void uncaughtException(Thread t, Throwable e)
```

You can install a handler into any thread with the `setUncaughtExceptionHandler` method. You can also install a default handler for all threads with the static method `setDefaultUncaughtExceptionHandler` of the `Thread` class. A replacement handler might use the logging API to send reports of uncaught exceptions into a log file. If you don't install a default handler, the default handler is `null`.

If you don't install an uncaught exception handler for a thread, it uses a handler that takes the following actions:

1. If the thread's thread group has a parent, then the `uncaughtException` method of the parent group is called. (This is not common.)
2. Otherwise, if the `Thread.getDefaultUncaughtExceptionHandler` method returns a non-`null` handler, it is called.

3. Otherwise, if the `Throwable` is an instance of the deprecated `ThreadDeath` class (which was once used to stop a thread), nothing happens.
4. Otherwise, the name of the thread and the stack trace of the `Throwable` are printed on `System.err`.

That is the stack trace that you have undoubtedly seen many times in your programs.



Note: Technically, the uncaught exception handler is defined by a thread's *thread group*. A thread group is a collection of threads that can be managed together. By default, all threads that you create belong to the same thread group, but it is possible to establish other groupings. Since there are now better features for operating on collections of threads, I recommend that you do not use thread groups in your programs.

`java.lang.Thread` **1.0**

- `static void setDefaultUncaughtExceptionHandler(Thread.UncaughtExceptionHandler handler)` **5.0**
- `static Thread.UncaughtExceptionHandler getDefaultUncaughtExceptionHandler()` **5.0**
  set or get the default handler for uncaught exceptions.
- `void setUncaughtExceptionHandler(Thread.UncaughtExceptionHandler handler)` **5.0**
- `Thread.UncaughtExceptionHandler getUncaughtExceptionHandler()` **5.0**
  set or get the handler for uncaught exceptions. If no handler is installed, the thread group object is the handler.

`java.lang.Thread.UncaughtExceptionHandler` **5.0**

- `void uncaughtException(Thread t, Throwable e)`
  defined to log a custom report when a thread is terminated with an uncaught exception.

`java.lang.ThreadGroup` **1.0**

- `void uncaughtException(Thread t, Throwable e)`
  calls this method of the parent thread group if there is a parent, or calls the default handler of the `Thread` class if there is a default handler, or otherwise prints

a stack trace to the standard error stream.

10.3.6. Thread Priorities

Every thread has a *priority.* By default, a thread inherits the priority of the thread that constructed it. You can increase or decrease the priority of any platform thread with the `setPriority` method. You can set the priority to any value between `MIN_PRIORITY` (defined as `1` in the `Thread` class) and `MAX_PRIORITY` (defined as `10`). `NORM_PRIORITY` is defined as `5`.

Whenever the platform thread scheduler has a chance to pick a new thread, it prefers threads with higher priority. However, thread priorities are *highly system-dependent*. Since the Java virtual machine relies on the thread implementation of the host platform, the Java thread priorities are mapped to the priority levels of the host platform, which may have more or fewer thread priority levels.

For example, Windows has seven priority levels. Some of the Java priorities will map to the same operating system level. In the OpenJDK VM for Linux, thread priorities are ignored altogether—all threads have the same priority.

Thread priorities may have been useful in early versions of Java that didn't use operating systems threads. You should not use them nowadays.

For backwards compatibility, all virtual threads have priority `NORM_PRIORITY`. Attempting to change the priority has no effect.

java.lang.Thread **1.0**

- `void setPriority(int newPriority)`
  sets the priority of this thread. The priority must be between `Thread.MIN_PRIORITY` and `Thread.MAX_PRIORITY`. Use `Thread.NORM_PRIORITY` for normal priority.
- `static int MIN_PRIORITY`
  is the minimum priority that a `Thread` can have. The minimum priority value is `1`.
- `static int NORM_PRIORITY`
  is the default priority of a `Thread`. The default priority is `5`.
- `static int MAX_PRIORITY`
  is the maximum priority that a `Thread` can have. The maximum priority value is `10`.

10.3.7. Thread Factories and Builders

You often want to produce multiple threads with the same properties. The `ThreadFactory` interface has a single method

```
Thread newThread(Runnable r)
```

that produces a new thread instance which, when started, will execute the given `Runnable`.

You can provide your own class that implements this interface, or, since Java 21, use a convenient builder syntax.

The static methods `Thread.ofPlatform()` and `Thread.ofVirtual()` yield builders for platform or virtual threads. You can customize the threads to be built:

```
Thread.Builder builder = Thread.ofVirtual().name("request-", 1);
```

See the API notes for additional customization methods.

Then call either `unstarted` or `start` to get a single thread, or `factory` to get a thread factory:

```
Thread t = builder.unstarted(myRunnable);
builder.start(myRunnable);

ThreadFactory factory = builder.factory();
t = factory.newThread(myRunnable);
```

`java.util.concurrent.ThreadFactory` **5.0**

- `Thread newThread(Runnable r)`
  creates a thread that, when started, calls the `run` method of the given `Runnable`.

java.lang.Thread 1.0

- Thread.Builder.OfPlatform ofPlatform() 21
- Thread.Builder.OfVirtual ofVirtual() 21

  create a builder for platform or virtual threads.

java.lang.Thread.Builder 21

- Thread.Builder name(String prefix, long start)
- Thread.Builder name(String name)

  cause the built threads to be named with the given prefix and a counter with the given start value, or with the given name.
- ThreadFactory factory()

  yields a factory for building threads with the configuration of this builder.
- Thread unstarted(Runnable task)
- Thread start(Runnable task)

  construct a thread executing the given task. The second method starts the thread.
- Thread.Builder uncaughtExceptionHandler(Thread.UncaughtExceptionHandler ueh)

  causes the built threads to use the given uncaught exception handler.

java.lang.Thread.Builder.OfPlatform 21

- Thread.Builder.OfPlatform daemon()
- Thread.Builder.OfPlatform daemon(boolean on)

  cause the built threads to be daemon threads unless `on` is `false`.
- Thread.Builder.OfPlatform stackSize(long stackSize)
- Thread.Builder.OfPlatform group(ThreadGroup group)
- Thread.Builder.OfPlatform priority(int priority)

  cause the built threads to have the given stack size hint, group, or priority (not recommended).

10.4. Coordinating Tasks

It is common to break tasks into subtasks whose results are combined when they are available. In the following sections, you will see the tools that the Java concurrency framework provides for coordinating concurrent tasks.

10.4.1. Callables and Futures

A `Runnable` encapsulates a task that runs asynchronously; you can think of it as an asynchronous method with no parameters and no return value. A `Callable` is similar to a `Runnable`, but it returns a value. The `Callable` interface is a parameterized type, with a single method `call`.

```
public interface Callable<V>
{
  V call() throws Exception;
}
```

The type parameter is the type of the returned value. For example, a `Callable<Integer>` represents an asynchronous task that eventually returns an `Integer` object.

A `Future` holds the *result* of an asynchronous task. When you schedule a task for execution, you receive a `Future` object. Using the `Future` object, you can obtain the result when it is ready.

The `Future<V>` interface has the following methods:

```
V get()
V get(long timeout, TimeUnit unit)
V resultNow()
Throwable exceptionNow()
void cancel(boolean mayInterrupt)
boolean isCancelled()
boolean isDone()
Future.State state()
```

A call to the first `get` method blocks until the task is finished. The second `get` method also blocks, but it throws a `TimeoutException` if the call timed out before the task finished. If the thread running the task is interrupted, both methods throw an `InterruptedException`. If the task was terminated with an exception, the `get` methods throw an `ExecutionException` whose cause is the terminating exception.

If the task has already finished, `get` returns immediately.

The `resultNow` method is a non-blocking version of `get` that you should only call if you know that the task has completed successfully. If you know that the task has failed, you can retrieve the terminating exception with the `exceptionNow` method.

You can attempt to cancel the task with the `cancel` method. If the task has not yet started, it is canceled and will never start. If the task is currently in progress, it is interrupted if the `mayInterrupt` parameter is `true`, and otherwise runs to completion.



Caution: Task cancelation in Java always requires cooperation from the task implementor. As described in Section 10.3.2, the task needs to monitor the interrupted status of the thread.

The `isDone` method returns `true` if the task has completed (successfully or unsuccessfully), or it has been canceled.

The `state` method returns the task state as a value of the `Task.State` enumeration with four values:

- `RUNNING`: The task has not completed and was not cancelled
- `SUCCESS`: The task has completed successfully
- `FAILED`: The task has completed with an exception
- `CANCELLED`: The task was cancelled

You should call the `state` method before calling `resultNow` or `exceptionNow`.

One way to execute a `Callable` is to use a `FutureTask`, which implements both the `Future` and `Runnable` interfaces, so that you can construct a thread for running it:

```
Callable<Integer> task = . . .;
var futureTask = new FutureTask<Integer>(task);
var t = new Thread(futureTask); // futureTask is a Runnable
t.start();
. . .
Integer result = futureTask.get(); // futureTask is a Future
```

More commonly, you will pass a `Callable` to an executor. That is the topic of the next section.

To get a `Future` for a `Runnable`, you can call:

```
var futureTask = new FutureTask<Void>(myRunnable, null);
```

java.util.concurrent.Callable<V> **5.0**

- `V call()`
  runs a task that yields a result.

java.util.concurrent.Future<V> **5.0**

- `V get()`
- `V get(long time, TimeUnit unit)`
  get the result, blocking until it is available or the given time has elapsed. Throw a `CancellationException` if the task was canceled, an `ExecutionException` whose cause is the terminating exception if the task completed unsuccessfully, and an `InterruptedException` if the current thread was interrupted. The second method throws a `TimeoutException` if the task was not done within the given timeout.
- `V resultNow()` **19**
  gets the result if the task completed successfully, throws an `IllegalStateException` otherwise.
- `Throwable exceptionNow()` **19**
  gets the terminating exception if the task completed unsuccessfully, throws an `IllegalStateException` otherwise.
- `boolean cancel(boolean mayInterrupt)`
  attempts to cancel the execution of this task. If the task has already started and the `mayInterrupt` parameter is `true`, it is interrupted. Returns `true` if the cancellation was successful.
- `boolean isCancelled()`
  returns `true` if the task was canceled before it completed.
- `boolean isDone()`
  returns `true` if the task completed, successfully or unsuccessfully, or was canceled.
- `Future.State state()` **19**
  returns the state of this task, as `RUNNING`, `SUCCESS`, `FAILED`, or `CANCELLED`.

java.util.concurrent.FutureTask<V> **5.0**

- `FutureTask(Callable<V> task)`
  constructs an object that is both a `Future<V>` and a `Runnable`.

- `FutureTask(Runnable task, V result)`
  constructs an object that, as a `Runnable`, executes the task, and as a `Future<V>`, yields the result upon completion.

10.4.2. Executors

Constructing a new platform thread is somewhat expensive because it involves interaction with the operating system. If your program creates a large number of short-lived platform threads, you should not run each task on a separate thread, but use a *thread pool* instead. A thread pool contains a number of threads that are ready to execute your tasks. You submit `Runnable` or `Callable` tasks to the pool, and one of the threads calls the `run` or `call` method. When the method exits, the thread doesn't die but stays around to serve the next request.

An *executor* is an object to which you submit tasks. For platform threads, that is a thread pool. Virtual threads are inexpensive to create and should never be pooled. An executor for virtual threads will simply start a new virtual thread for each submitted task.

The `Executors` class has a number of static factory methods for constructing executors. **Table 10.1** lists the most useful ones.

Table 10.1: Common Executors Factory Methods

Method	Description
`newCachedThreadPool`	New threads are created as needed; idle threads are kept for 60 seconds.
`newFixedThreadPool`	The pool contains a fixed set of threads; idle threads are kept indefinitely.
`newSingleThreadExecutor`	A "pool" with a single thread that executes the submitted tasks sequentially.
`newVirtualThreadPerTaskExecutor`	An executor that runs each task on a new virtual thread.

The `newCachedThreadPool` method constructs a thread pool that executes each task immediately, using an existing idle thread when available and creating a new thread otherwise. The `newFixedThreadPool` method constructs a thread pool with a fixed size. If more

tasks are submitted than there are idle threads, the unserved tasks are placed on a queue. They are run when other tasks have completed. The `newSingleThreadExecutor` is a degenerate pool of size 1 where a single thread executes the submitted tasks, one after another.

Use a cached thread pool when you have platform threads that are short-lived or spend a lot of time blocking. However, if you have threads that are working hard without blocking, you don't want to run a large number of them together. For optimum speed, the number of concurrent threads is the number of processor cores. In such a situation, you should use a fixed thread pool that bounds the total number of concurrent platform threads.

Another reason for using a fixed thread pool is to throttle the number of concurrent tasks. If many of the tasks access a service that can only handle a limited number of concurrent clients, then you can use the pool size to enforce that limit. This is a rather crude way of tuning an application, but it is surprisingly common because it is easy to do.



Caution: If your application uses virtual threads, you need to come up with alternative mechanisms for throttling concurrent requests, such as a counting semaphore or bounded queue.

For example, this code snippet limits the number of concurrent web requests:

```
private static final int CONCURRENT_REQUESTS = 200;
private static final Semaphore SEMAPHORE = new Semaphore(CONCURRENT_REQUESTS);
...
public static String get(URI uri) throws InterruptedException, IOException {
  var request = HttpRequest.newBuilder().uri(uri).GET().build();
  SEMAPHORE.acquire();
  try {
    return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
  } finally {
    SEMAPHORE.release();
  }
}
```

With a high number of concurrent requests, the `acquire` method blocks. For virtual threads, that is not a problem. For effective scaling, it is better to move the blocking to the point of pressure instead of the thread pool.

The single-thread executor is potentially useful for performance analysis. If you temporarily replace a cached or fixed thread pool with a single-thread pool, you can measure how much slower your application runs without the benefit of concurrency.

Most of the `Executors` factory methods have a version with a `ThreadFactory` parameter, allowing you to customize the threads that execute the submitted tasks. If you provide your own factory for a thread pool, it should produce platform threads, not virtual threads, since pooling virtual threads is never useful. If you want to customize virtual threads, pass a factory for virtual threads to the `newThreadPerTaskExecutor` method.

All of the `Executors` factory methods return objects of classes that implement the `ExecutorService` interface. Submit a `Runnable` or `Callable` to an `ExecutorService` with one of the following methods:

```
Future<T> submit(Callable<T> task)
Future<?> submit(Runnable task)
Future<T> submit(Runnable task, T result)
```

When you call `submit`, the executor will schedule the submitted task. You get back a `Future` object that you can use to get the result or cancel the task.

The second `submit` method returns a `Future<?>`. The `get` method blocks until completion and then returns `null`.



Note: Other methods in the `java.util.concurrent` API return a `Future<Void>` instead of a `Future<?>`. For all practical purposes these types have the same function. You can use instances to query the task state and to block until the task completes, but there is no result value.

The third version of `submit` yields a `Future` whose `get` method returns the given `result` object upon completion.

When you are done with an executor, call the `close` method. An executor that is closed down accepts no new tasks. The `close` method blocks until all submitted tasks are finished. (If the thread calling `close` is interrupted, then the submitted tasks are canceled.) A convenient way to invoke `close` is to use a try-with-resources statement:

```
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
   . . .
   Future<V> f = submit(myCallable);
   . . .
} // executor.close() called here
```



Note: Some programmers view an executor as a global resource. There are situations where a global executor makes sense, but you should not hesitate to have local executors, submit a few tasks, and then close them. This is particularly useful with virtual threads, where the blocking call to `close` is not a concern.

There are other methods to shut down an executor: `shutdown`, `shutdownNow`, and `awaitTermination`. These give you some finer-grained control, and they are needed in legacy code since the `close` method was only added in Java 19. If you need to run the demo program in this section with an older version of Java, replace the call `executor.close()` with:

```
executor.shutdown();
executor.awaitTermination(10, TimeUnit.MINUTES);
```

Here, in summary, is what you do to use an executor:

1. Call one of the static methods of the `Executors` class, most commonly `newCachedThreadPool`, `newFixedThreadPool`, or `newVirtualThreadPerTaskExecutor`.
2. Call `submit` to submit `Callable` or `Runnable` objects.
3. Hang on to the returned `Future` objects so that you can query task states, get the results, or cancel the tasks.
4. Shut down the executor when you no longer want to submit any tasks.



Caution: When an executor uses threads from a thread factory, setting the factory's uncaught exception handler has no effect. Thread pools need to install their own handler for catching task exceptions.

java.util.concurrent.Executors 5.0

- static ExecutorService newCachedThreadPool()
- static ExecutorService newCachedThreadPool(ThreadFactory threadFactory)
  returns a cached thread pool that creates threads as needed and terminates threads that have been idle for 60 seconds.
- static ExecutorService newFixedThreadPool(int threads)
- static ExecutorService newFixedThreadPool(int threads, ThreadFactory threadFactory)
  returns a thread pool that uses the given number of threads to execute tasks.
- static ExecutorService newSingleThreadExecutor()
- static ExecutorService newSingleThreadExecutor(ThreadFactory threadFactory)
  returns an executor that executes tasks sequentially in a single thread.
- static ExecutorService newVirtualThreadPerTaskExecutor() 21
- static ExecutorService newThreadPerTaskExecutor(ThreadFactory threadFactory) 21
  returns an executor that executes each task in a new virtual thread, or a thread produced by the given factory.

java.util.concurrent.ExecutorService 5.0

- Future<T> submit(Callable<T> task)
- Future<T> submit(Runnable task, T result)
- Future<?> submit(Runnable task)
  submit the given task for execution.
- void close() 19
  sets this executor into shutdown mode where it no longer accepts new submissions, and blocks until the already submitted tasks have completed. If interrupted while blocking, running tasks are interrupted.
- void shutdown()
  sets this executor into shutdown mode where it no longer accepts new submissions. This method does not block for completion of submitted tasks.
- List<Runnable> shutdownNow()
  sets this executor into shutdown mode, interrupts tasks that have started, and removes tasks that have not yet started. Returns a list of the removed tasks. This method does not block for completion of submitted tasks.

- `boolean awaitTermination(long timeout, TimeUnit unit)`
  blocks until all submitted tasks have completed, the timeout has elapsed, or the blocking is interrupted. Returns `true` if all tasks have completed.

10.4.3. Invoking a Group of Tasks

Often, you want to launch multiple concurrent tasks and get the result of the first successful one, or combine the results of all completed ones.

The `invokeAny` method submits all tasks in a collection of `Callable` objects and blocks until one of them has completed successfully, returning its result. Any tasks that have not completed are canceled. You don't know which task succeeded—presumably, it is the one that finished most quickly. If no task succeeded, an `ExecutionException` is thrown.

Use this method for a search problem in which you are willing to accept any solution. For example, suppose that you need to find a factor of a large integer—a computation that is required for breaking the RSA cipher. You could submit a number of tasks, each attempting a factorization with numbers in a different range. As soon as one of these tasks has an answer, your computation can stop.

The `invokeAll` method submits all tasks in a collection of `Callable` objects, blocks until all of them complete, and returns a list of `Future` objects that represent the solutions to all tasks, in the same order as the submitted tasks. You can process the results of the computation like this:

```
List<Callable<T>> tasks = . . .;
List<Future<T>> results = executor.invokeAll(tasks);
for (Future<T> result : results) {
  if (result.state() == Future.State.SUCCESS)
    process result.resultNow();
  else
    handle result.exceptionNow();
}
```

The call to `invokeAll` blocks until all tasks have completed. If you want to obtain the results in the order in which they are available, use an `ExecutorCompletionService`.

Start with an executor, obtained in the usual way. Then construct an `ExecutorCompletionService`. Submit tasks to the completion service. The service manages

a blocking queue of `Future` objects, containing the results of the submitted tasks as they become available:

```
var service = new ExecutorCompletionService<T>(executor);
for (Callable<T> task : tasks) service.submit(task);
for (int i = 0; i < tasks.size(); i++) {
  Future<T> result = service.take();
}
```

The program in **Listing 10.3** shows how to use callables and executors. In the first computation, we count how many files in a directory tree contain a given word. We make a separate task for each file:

```
Set<Path> files = descendants(Path.of(start));
var tasks = new ArrayList<Callable<Long>>();
for (Path file : files)
{
  Callable<Long> task = () -> occurrences(word, file);
  tasks.add(task);
}
```

Then we pass the tasks to an executor service:

```
ExecutorService executor = Executors.newCachedThreadPool();
List<Future<Long>> results = executor.invokeAll(tasks);
```

To get the combined count, we add all results, blocking until they are available:

```
long total = 0;
for (Future<Long> result : results)
  total += result.get();
```

The program also displays the time spent during the search. Unzip the source code for the JDK somewhere and run the search. Then replace the executor service with a single-thread executor and try again to see whether the concurrent computation was faster. Also try with virtual threads.

In the second part of the program, we search for the first file that contains the given word. We use `invokeAny` to parallelize the search. Here, we have to be more careful about formulating the tasks. The `invokeAny` method terminates as soon as any task *returns*. So we cannot have the search tasks return a `boolean` to indicate success or failure. We don't want to stop searching when a task failed. Instead, a failing task throws a `NoSuchElementException`. Also, when one task has succeeded, the others are canceled. Therefore, we monitor the interrupted status. If the underlying thread is interrupted, the search task prints a message before terminating, so that you can see that the cancellation is effective.

```
public static Callable<Path> searchForTask(String word, Path path)
{
  return () ->
    {
      try (var in = new Scanner(path))
      {
        while (in.hasNext())
        {
          if (in.next().equals(word)) return path;
          if (Thread.currentThread().isInterrupted())
          {
            System.out.println("Search in " + path + " canceled.");
            return null;
          }
        }
        throw new NoSuchElementException();
      }
    };
}
```



Preview Note: Just like structured programming replaced “goto” statements with branches, loops, and functions, *structured concurrency* is an effort to find programming constructs that make it easier to reason about concurrent programs.

Java 21 contains a preview of a structured concurrency API. Tasks are submitted to a `StructuredTaskScope`. Then the `join` method blocks until the scope is *shut down*, according to the scope's shutdown policy. The API provides two simple policies `ShutdownOnFailure` and `ShutdownOnSuccess`. Here is a usage example:

```
// Java 21 preview API
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
   Supplier<T1> f1 = scope.fork(callable1);
   Supplier<T2> f2 = scope.fork(callable2);
   scope.join();
   scope.throwIfFailed();
   result = combine(f1.get(), f2.get());
}
```

When the `try` block ends, we can be sure that all subtasks have completed.

If either subtask fails, the scope shuts down, canceling the other task, and the first exception is rethrown.

If the current thread is interrupted, the scope is closed and both tasks are canceled.

You could have achieved almost the same effect with `ExecutorService.invokeAll`, except that `invokeAll` waits for completion of all tasks even if one of them fails. If you prefer that policy, it is easy to implement a scope subclass.

The structured programming API is flexible and convenient, and it provides runtime checks and observability features for the nesting of scopes.

Listing 10.3 executors/ExecutorDemo.java

```
1  package executors;
2
3  import java.io.*;
4  import java.nio.file.*;
5  import java.time.*;
6  import java.util.*;
7  import java.util.concurrent.*;
8  import java.util.stream.*;
9
10 /**
11  * This program demonstrates the Callable interface and executors.
12  * @version 1.02 2023-07-27
13  * @author Cay Horstmann
14  */
15 public class ExecutorDemo
16 {
17    /**
```

```
18    * Counts occurrences of a given word in a file.
19    * @return the number of times the word occurs in the given word
20    */
21   public static long occurrences(String word, Path path)
22   {
23     try (var in = new Scanner(path))
24     {
25       int count = 0;
26       while (in.hasNext())
27         if (in.next().equals(word)) count++;
28       return count;
29     }
30     catch (IOException ex)
31     {
32       return 0;
33     }
34   }
35
36   /**
37    * Returns all descendants of a given directory--see Chapters 1 and 2 of Volume II.
38    * @param rootDir the root directory
39    * @return a set of all descendants of the root directory
40    */
41   public static Set<Path> descendants(Path rootDir) throws IOException
42   {
43     try (Stream<Path> entries = Files.walk(rootDir))
44     {
45       return entries.filter(Files::isRegularFile)
46         .collect(Collectors.toSet());
47     }
48   }
49
50   /**
51    * Yields a task that searches for a word in a file.
52    * @param word the word to search
53    * @param path the file in which to search
54    * @return the search task that yields the path upon success
55    */
56   public static Callable<Path> searchForTask(String word, Path path)
57   {
58     return () ->
59       {
60         try (var in = new Scanner(path))
61         {
62           while (in.hasNext())
```

```
63            {
64              if (in.next().equals(word)) return path;
65              if (Thread.currentThread().isInterrupted())
66              {
67                System.out.println("Search in " + path + " canceled.");
68                return null;
69              }
70            }
71            throw new NoSuchElementException();
72          }
73        };
74    }
75
76    public static void main(String[] args)
77        throws InterruptedException, ExecutionException, IOException
78    {
79      try (var in = new Scanner(System.in))
80      {
81        System.out.print("Enter base directory (e.g. /opt/jdk-21-src): ");
82        String start = in.nextLine();
83        System.out.print("Enter keyword (e.g. volatile): ");
84        String word = in.nextLine();
85
86        Set<Path> files = descendants(Path.of(start));
87        var tasks = new ArrayList<Callable<Long>>();
88        for (Path file : files)
89        {
90          Callable<Long> task = () -> occurrences(word, file);
91          tasks.add(task);
92        }
93        ExecutorService executor = Executors.newCachedThreadPool();
94        // use a single thread executor instead to see if multiple threads
95        // speed up the search
96        // ExecutorService executor = Executors.newSingleThreadExecutor();
97        // Or try virtual threads
98        // ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
99        Instant startTime = Instant.now();
100       List<Future<Long>> results = executor.invokeAll(tasks);
101       Instant endTime = Instant.now();
102       long total = 0;
103       for (Future<Long> result : results)
104         total += result.get();
105       System.out.println("Occurrences of " + word + ": " + total);
106       System.out.println("Time elapsed: "
107         + Duration.between(startTime, endTime).toMillis() + " ms");
```

```
108
109         var searchTasks = new ArrayList<Callable<Path>>();
110         for (Path file : files)
111           searchTasks.add(searchForTask(word, file));
112         startTime = Instant.now();
113         Path found = executor.invokeAny(searchTasks);
114         endTime = Instant.now();
115         System.out.println(word + " occurs in: " + found);
116         System.out.println("Time elapsed: "
117           + Duration.between(startTime, endTime).toMillis() + " ms");
118
119         executor.close();
120       }
121     }
122 }
```

java.util.concurrent.ExecutorService **5.0**

- `T invokeAny(Collection<Callable<T>> tasks)`
- `T invokeAny(Collection<Callable<T>> tasks, long timeout, TimeUnit unit)`
  execute the given tasks and return the result of one that completed successfully, canceling the remaining tasks. The second method throws a `TimeoutException` if a timeout occurs.
- `List<Future<T>> invokeAll(Collection<Callable<T>> tasks)`
- `List<Future<T>> invokeAll(Collection<Callable<T>> tasks, long timeout, TimeUnit unit)`
  execute the given tasks and return the results when all tasks have completed. The second method throws a `TimeoutException` if a timeout occurs.

java.util.concurrent.ExecutorCompletionService<V> **5.0**

- `ExecutorCompletionService(Executor e)`
  constructs an executor completion service that collects the results of the given executor.
- `Future<V> submit(Callable<V> task)`
- `Future<V> submit(Runnable task, V result)`
  submit a task to the underlying executor.
- `Future<V> take()`
  removes the next completed result, blocking if no completed results are available.

- `Future<V> poll()`
- `Future<V> poll(long time, TimeUnit unit)`

  remove and return the next completed result, or returns `null` if no completed results are available. The second method waits for the given time.

10.4.4. Thread-Local Variables

Sometimes you want to make a task-specific object available to all methods that collaborate on the task, without having to pass the object as a parameter of each method call. You can use a *thread-local variable* for this purpose. This is not an actual variable, but an object whose `get` and `set` methods access a value that depends on the current thread.

Suppose, for example, that you want to share a database connection. Declare a variable

```
public static final ThreadLocal<Connection> CONNECTION = new ThreadLocal<>();
```

When the task starts, initialize the connection for this thread:

```
CONNECTION.set(connect(uri, username, password));
```

The task calls some methods, all within the same thread, and eventually one of them needs the connection:

```
Connection connection = CONNECTION.get();
var result = connection.executeQuery(query);
```

Note that the same call may happen on multiple threads. Each of them gets its own connection object.



Caution: When executing tasks with a thread pool, you don't want to make thread-local data available to the other tasks that are scheduled on the same thread. You also want to make sure they get garbage collected when your task is done. It is essential that you call

```
CONNECTION.remove();
```

upon task completion (successfully or with an exception).

An `InheritableThreadLocal` is a subclass of `ThreadLocal`, used to propagate thread-local data to subtasks. When a new thread is started, it receives a copy of the parent's inheritable thread locals. (The copy is necessary because the child thread might mutate the thread-local values.)



Tip: Sharing resource-intensive objects as thread locals can be a problem when migrating to virtual threads. There will likely be far more of them than threads in a thread pool, and now you have many more instances. In such a situation, you should rethink your sharing strategy.

To identify the use of thread-local variables, run with the VM flag `jdk.traceVirtualThreadLocals`. You will get a stack trace when a virtual thread mutates a thread-local variable.



Preview Note: Scoped values are a preview feature of Java 21 that provide a more performant version of inheritable thread-local variables. Scoped values have per-thread instances, but they are immutable and have a bounded lifetime. Scoped values are inherited in virtual threads created by a `StructuredTaskScope`. This code snippet shows how to use them:

```
public static final ScopedValue<Connection> CONNECTION = ScopedValue.newInstance();
. . .
ScopedValue.where(CONNECTION, connect(uri, username, password)).run(() -> doWork());
. . .
public void doWork() {
  . . .
  Connection connection = CONNECTION.get();
  var result = connection.executeQuery(query);
  . . .
}
```

When using virtual threads, prefer scoped values over thread-local variables.

`java.lang.ThreadLocal<T>` **1.2**

- `T get()`
  gets the current value of this thread. If `get` is called for the first time and `set` was never called, the value is obtained by calling `initialize`.

- `void set(T t)`
  sets a new value for this thread.
- `void remove()`
  removes the value for this thread.

10.4.5. The Fork-Join Framework

Some applications use a large number of threads that are mostly idle. An example would be a web server that uses one thread per connection. Other applications use one thread per processor core, in order to carry out computationally intensive tasks, such as image or video processing. The fork-join framework is designed to support the latter. Suppose you have a processing task that naturally decomposes into subtasks, like this:

```
if (problemSize < threshold)
{
  solve problem directly
}
else
{
  break problem into subproblems
  recursively solve each subproblem
  combine the results
}
```

One example is image processing. To enhance an image, you can transform the top half and the bottom half. If you have enough idle processors, those operations can run in parallel. (You will need to do a bit of extra work along the strip that separates the two halves, but that's a technical detail.)

Here, we discuss a simpler example. Suppose we want to count how many elements of an array fulfill a particular property. We cut the array in half, compute the counts of each half, and add them up.

To put the recursive computation in a form that is usable by the framework, supply a class that extends `RecursiveTask<T>` (if the computation produces a result of type `T`) or `RecursiveAction` (if it doesn't produce a result). Override the `compute` method to generate and invoke subtasks and to combine their results.

```
class Counter extends RecursiveTask<Integer>
{
  . . .
  protected Integer compute()
  {
    if (to - from < THRESHOLD)
    {
      solve problem directly
    }
    else
    {
      int mid = from + (to - from) / 2;
      var first = new Counter(values, from, mid, filter);
      var second = new Counter(values, mid, to, filter);
      invokeAll(first, second);
      return first.join() + second.join();
    }
  }
}
```

The `invokeAll` method receives a number of tasks and blocks until all of them have completed. The `join` method yields the result. Here, we apply `join` to each subtask and return the sum.



Note: There is also a `get` method for getting the current result, but it is less attractive since it can throw checked exceptions that we are not allowed to throw in the `compute` method.

Listing 10.4 shows the complete example.

Behind the scenes, the fork-join framework uses an effective heuristic, called *work stealing*, for balancing the workload among available threads. Each worker thread has a deque (double-ended queue) for tasks. A worker thread pushes smaller subtasks onto the head of its own deque. (Only one thread accesses the head, so no locking is required.) When a worker thread is idle, it "steals" a task from the tail of another deque (which requires locking). By stealing larger subtasks, which are at the tail, more time elapses until the next "theft", reducing contention at the tails.



Caution: Fork-join pools are optimized for nonblocking workloads. If you add many blocking tasks into a fork-join pool, you can starve it. It is possible to overcome this

by having tasks implement the `ForkJoinPool.ManagedBlocker` interface, but this is an advanced technique that we won't discuss.

Listing 10.4 forkJoin/ForkJoinTest.java

```
1  package forkJoin;
2
3  import java.util.concurrent.*;
4  import java.util.function.*;
5
6  /**
7   * This program demonstrates the fork-join framework.
8   * @version 1.02 2021-06-17
9   * @author Cay Horstmann
10  */
11 public class ForkJoinTest
12 {
13    public static void main(String[] args)
14    {
15       final int SIZE = 10000000;
16       var numbers = new double[SIZE];
17       for (int i = 0; i < SIZE; i++) numbers[i] = Math.random();
18       var counter = new Counter(numbers, 0, numbers.length, x -> x > 0.5);
19       var pool = new ForkJoinPool();
20       pool.invoke(counter);
21       System.out.println(counter.join());
22    }
23 }
24
25 class Counter extends RecursiveTask<Integer>
26 {
27    public static final int THRESHOLD = 1000;
28    private double[] values;
29    private int from;
30    private int to;
31    private DoublePredicate filter;
32
33    public Counter(double[] values, int from, int to, DoublePredicate filter)
34    {
35       this.values = values;
36       this.from = from;
37       this.to = to;
38       this.filter = filter;
39    }
40
```

```
41    protected Integer compute()
42    {
43       if (to - from < THRESHOLD)
44       {
45          int count = 0;
46          for (int i = from; i < to; i++)
47          {
48             if (filter.test(values[i])) count++;
49          }
50          return count;
51       }
52       else
53       {
54          int mid = from + (to - from) / 2;
55          var first = new Counter(values, from, mid, filter);
56          var second = new Counter(values, mid, to, filter);
57          invokeAll(first, second);
58          return first.join() + second.join();
59       }
60    }
61 }
```

10.5. Synchronization

In most practical multithreaded applications, two or more threads need to share access to the same data. What happens if two threads have access to the same object and each calls a method that modifies the state of the object? As you might imagine, the threads can step on each other's toes. Depending on the order in which the data were accessed, corrupted objects can result. Such a situation is often called a *race condition.*

10.5.1. An Example of a Race Condition

To avoid corruption of shared data by multiple threads, you must learn how to *synchronize the access*. In this section, you'll see what happens if you do not use synchronization. In the next section, you'll see how to synchronize data access.

In the next test program, we continue working with our simulated bank. Unlike the example in **Section 10.1**, we randomly select the source and destination of the transfer. Since this will cause problems, let us look more carefully at the code for the `transfer` method of the `Bank` class.

```
public void transfer(int from, int to, double amount)
  // CAUTION: unsafe when called from multiple threads
{
  System.out.print(Thread.currentThread());
  accounts[from] -= amount;
  System.out.printf(" %10.2f from %d to %d", amount, from, to);
  accounts[to] += amount;
  System.out.printf(" Total Balance: %10.2f%n", getTotalBalance());
}
```

Here is the code for the `Runnable` instances. The `run` method keeps moving money out of a given bank account. In each iteration, the `run` method picks a random target account and a random amount, calls `transfer` on the bank object, and then sleeps.

```
Runnable r = () ->
  {
    try
    {
      while (true)
      {
        int toAccount = (int) (bank.size() * Math.random());
        double amount = MAX_AMOUNT * Math.random();
        bank.transfer(fromAccount, toAccount, amount);
        Thread.sleep((int) (DELAY * Math.random()));
      }
    }
    catch (InterruptedException e)
    {
    }
  };
```

When this simulation runs, we do not know how much money is in any one bank account at any time. But we do know that the total amount of money in all the accounts should remain unchanged because all we do is move money from one account to another.

At the end of each transaction, the `transfer` method recomputes the total and prints it.

This program never finishes. Just press Ctrl+C to kill the program.

Here is a typical printout:

```
. . .
Thread[Thread-11,5,main]    588.48 from 11 to 44 Total Balance:  100000.00
Thread[Thread-12,5,main]    976.11 from 12 to 22 Total Balance:  100000.00
Thread[Thread-14,5,main]    521.51 from 14 to 22 Total Balance:  100000.00
Thread[Thread-13,5,main]    359.89 from 13 to 81 Total Balance:  100000.00
. . .
Thread[Thread-36,5,main]    401.71 from 36 to 73 Total Balance:   99291.06
Thread[Thread-35,5,main]    691.46 from 35 to 77 Total Balance:   99291.06
Thread[Thread-37,5,main]     78.64 from 37 to 3 Total Balance:   99291.06
Thread[Thread-34,5,main]    197.11 from 34 to 69 Total Balance:   99291.06
Thread[Thread-36,5,main]     85.96 from 36 to 4 Total Balance:   99291.06
. . .
Thread[Thread-4,5,main]Thread[Thread-33,5,main]      7.31 from 31 to 32 Total Balance:
99979.24
    627.50 from 4 to 5 Total Balance:  99979.24
. . .
```

As you can see, something is very wrong. For a few transactions, the bank balance remains at $100,000, which is the correct total for 100 accounts of $1,000 each. But after some time, the balance changes slightly. When you run this program, errors may happen quickly, or it may take a very long time for the balance to become corrupted. This situation does not inspire confidence, and you would probably not want to deposit your hard-earned money in such a bank.

See if you can spot the problems with the code in **Listing 10.5** and the `Bank` class in **Listing 10.2**. We will unravel the mystery in the next section.

Listing 10.5 unsynch/UnsynchBankTest.java

```
1  package unsynch;
2
3  /**
4   * This program shows data corruption when multiple threads access a data structure.
5   * @version 1.33 2023-07-30
6   * @author Cay Horstmann
7   */
8  public class UnsynchBankTest
9  {
10   public static final int NACCOUNTS = 100;
11   public static final double INITIAL_BALANCE = 1000;
12   public static final double MAX_AMOUNT = 1000;
13   public static final int DELAY = 10;
```

```
14
15    public static void main(String[] args)
16    {
17       var bank = new Bank(NACCOUNTS, INITIAL_BALANCE);
18       for (int i = 0; i < NACCOUNTS; i++)
19       {
20          int fromAccount = i;
21          Runnable r = () ->
22             {
23                try
24                {
25                   while (true)
26                   {
27                      int toAccount = (int) (bank.size() * Math.random());
28                      double amount = MAX_AMOUNT * Math.random();
29                      bank.transfer(fromAccount, toAccount, amount);
30                      Thread.sleep((int) (DELAY * Math.random()));
31                   }
32                }
33                catch (InterruptedException e)
34                {
35                }
36             };
37          Thread.ofPlatform().start(r);
38       }
39    }
40 }
```

10.5.2. The Race Condition Explained

In the previous section, we ran a program in which several threads updated bank account balances. After a while, errors crept in and some amount of money was either lost or spontaneously created. This problem occurs when two threads are simultaneously trying to update an account. Suppose two threads simultaneously carry out the instruction

```
accounts[to] += amount;
```

The problem is that these are not *atomic* operations. The instruction might be processed as follows:

1. Load `accounts[to]` into a register.
2. Add `amount`.

3. Move the result back to `accounts[to]`.

Now, suppose the first thread executes Steps 1 and 2, and then it is preempted. Suppose the second thread awakens and updates the same entry in the `account` array. Then, the first thread awakens and completes its Step 3.

That action wipes out the modification of the other thread. As a result, the total is no longer correct (see **Figure 10.2**).

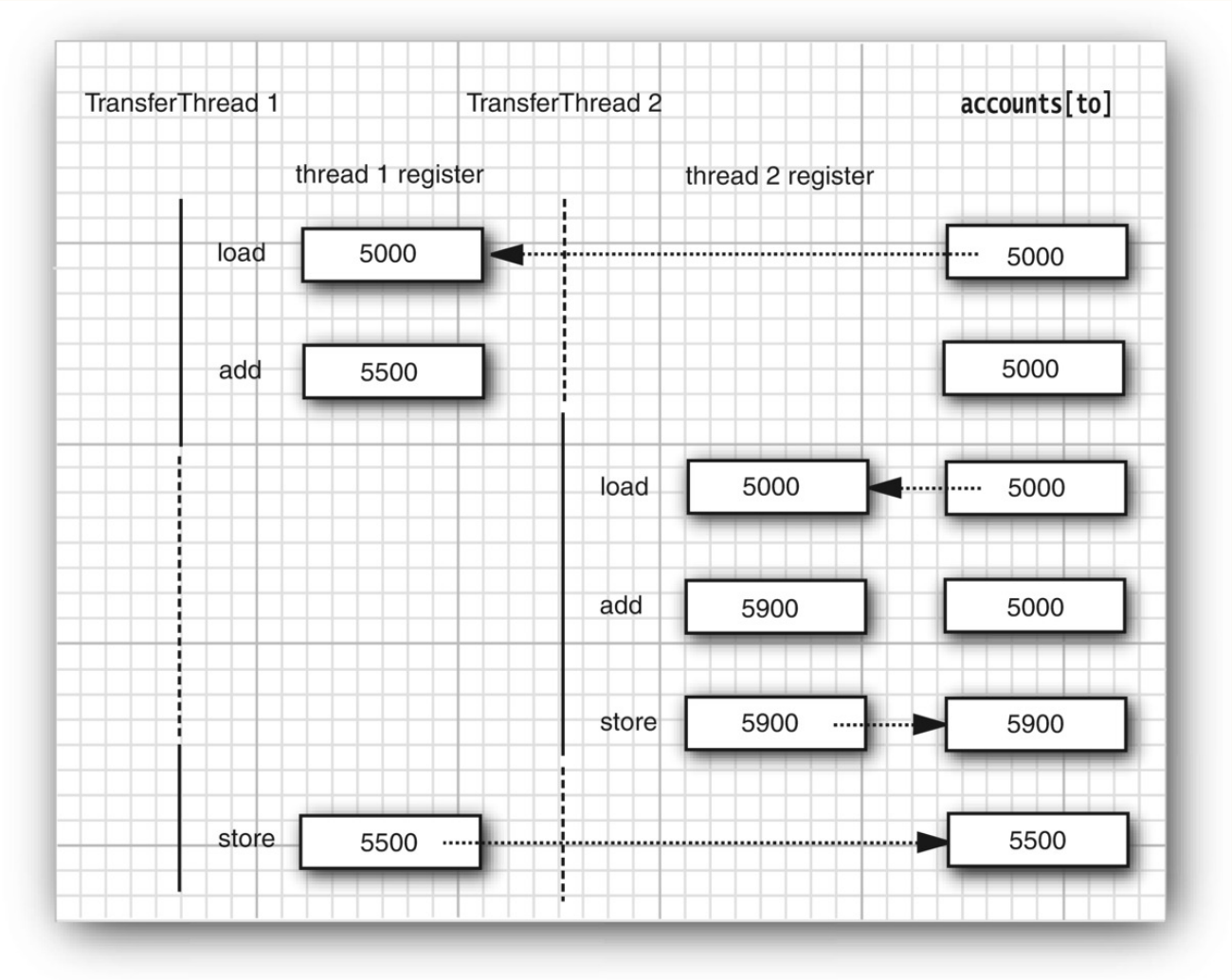


Figure 10.2: Simultaneous access by two threads

The `UnsyncBankTest` program demonstrates this corruption. (There is also a slight chance of false alarms during the computation of `getTotalBalance`.)

Note: You can actually peek at the virtual machine bytecodes that execute each statement in our class. Run the command

```
javap -c -v Bank
```

to decompile the `Bank.class` file. For example, the line

```
accounts[to] += amount;
```

is translated into the following bytecodes:

```
aload_0
getfield      #2; // Field accounts:[D
iload_2
dup2
daload
dload_3
dadd
dastore
```

What these codes mean does not matter. The point is that the increment command is made up of several instructions, and the thread executing them can be preempted at any instruction.

What is the chance of this corruption occurring? On a modern processor with multiple cores, the risk of corruption is quite high. I boosted the chance of observing the problem by interleaving the print statements with the statements that update the balance.

If you run lots of threads on a heavily loaded machine, the program will still fail even after you have eliminated the print statements. The failure may take a few seconds or many hours or to occur. Frankly, there are few things worse in the life of a programmer than an error that only manifests itself irregularly.

The real problem is that the work of the `transfer` method can be preempted in the middle. If we could ensure that the method runs to completion before the thread loses control, the state of the bank account object would never be corrupted.

10.5.3. Lock Objects

There are two mechanisms for protecting a code block from concurrent access. The Java language provides a `synchronized` keyword for this purpose, and the `java.util.concurrent` package provides explicit classes for locking. The `synchronized` keyword automatically provides a lock as well as an associated "condition," which makes it powerful and convenient for many cases that require explicit locking. However, I believe that it is easier to un-

derstand the `synchronized` keyword after you have seen locks and conditions in isolation. I explain the explicit classes here and in **Section 10.5.4**. Once you have understood these building blocks, you can read about the `synchronized` keyword in **Section 10.5.6**.

The basic outline for protecting a code block with a `ReentrantLock` is:

```
myLock.lock(); // a ReentrantLock object
try
{
  critical section
}
finally
{
  myLock.unlock(); // make sure the lock is unlocked even if an exception is thrown
}
```

This construct guarantees that only one thread at a time can enter the critical section. As soon as one thread locks the lock object, no other thread can get past the `lock` statement. When other threads call `lock`, they are deactivated until the first thread unlocks the lock object.



Caution: It is critically important that the `unlock` operation is enclosed in a `finally` clause. If the code in the critical section throws an exception, the lock must be unlocked. Otherwise, the other threads will be blocked forever.



Note: When you use locks, you cannot use the `try`-with-resources statement. The unlock method isn't called `close`. Some programmers proposed to make `Lock` extend `AutoCloseable`, with `close` calling `lock`. But Java designers felt that lock usage is fairly uncommon and deserves to be noticeable.

Let us use a lock to protect the `transfer` method of the `Bank` class.

```
public class Bank
{
  private final Lock bankLock = new ReentrantLock();
  . . .
  public void transfer(int from, int to, int amount)
  {
    bankLock.lock();
    try
    {
```

```
        System.out.print(Thread.currentThread());
        accounts[from] -= amount;
        System.out.printf(" %10.2f from %d to %d", amount, from, to);
        accounts[to] += amount;
        System.out.printf(" Total Balance: %10.2f%n", getTotalBalance());
      }
      finally
      {
        bankLock.unlock();
      }
    }
  }
```

Suppose one thread calls `transfer` and gets preempted before it is done. Suppose a second thread also calls `transfer`. The second thread cannot acquire the lock and is blocked in the call to the `lock` method. It is deactivated and must wait for the first thread to finish executing the `transfer` method. When the first thread unlocks the lock, then the second thread can proceed (see **Figure 10.3**).

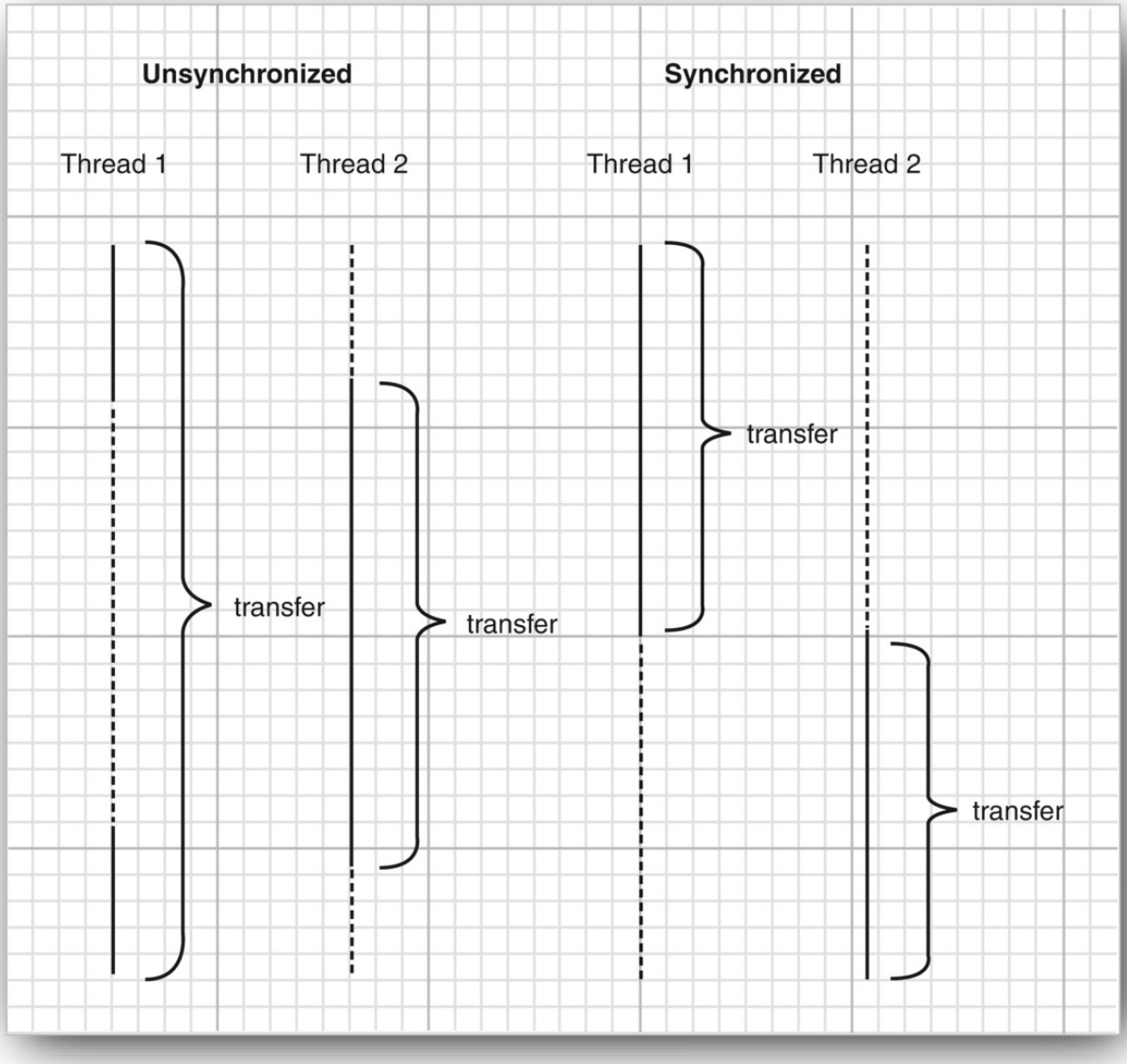


Figure 10.3: Comparison of unsynchronized and synchronized threads

Try it out. Add the locking code to the `transfer` method and run the program again. You can run it forever, and the bank balance will not become corrupted.

Note that each `Bank` object has its own `ReentrantLock` object. If two threads try to access the same `Bank` object, then the lock serves to serialize the access. However, if two threads access different `Bank` objects, each thread acquires a different lock and neither thread is blocked. This is as it should be, because the threads cannot interfere with one another when they manipulate different `Bank` instances.

The lock is called *reentrant* because a thread can repeatedly acquire a lock that it already owns. The lock has a *hold count* that keeps track of the nested calls to the `lock` method. The thread has to call `unlock` for every call to `lock` in order to relinquish the lock. Because of this feature, code protected by a lock can call another method that uses the same lock.

For example, the `transfer` method calls the `getTotalBalance` method, which also locks the `bankLock` object, which now has a hold count of `2`. When the `getTotalBalance` method exits, the hold count is back to `1`. When the `transfer` method exits, the hold count is `0`, and the thread relinquishes the lock.

In general, you will want to use a lock around any block of code that updates or inspects a shared object, so you can be assured that these operations run to completion before another thread can use the same object.



Caution: Be careful to ensure that the code in a critical section is not bypassed by throwing an exception. If an exception is thrown before the end of the section, the `finally` clause will relinquish the lock, but the object may be in a damaged state.

`java.util.concurrent.locks.Lock` **5.0**

- `void lock()`
  acquires this lock; blocks if the lock is currently owned by another thread.
- `void unlock()`
  releases this lock.

`java.util.concurrent.locks.ReentrantLock` **5.0**

- `ReentrantLock()`
  constructs a reentrant lock that can be used to protect a critical section.
- `ReentrantLock(boolean fair)`
  constructs a lock with the given fairness policy. A fair lock favors the thread that has been waiting for the longest time. However, this fairness guarantee can be a significant drag on performance. Therefore, by default, locks are not required to be fair.



Caution: It sounds nice to be fair, but fair locks can be *a lot slower* than regular locks. You should only enable fair locking if you truly know what you are doing and have a specific reason to consider fairness essential for your program. Even if you use a fair lock, you have no guarantee that the thread scheduler is fair. If the thread scheduler chooses to neglect a thread that has been waiting a long time for the lock, it doesn't get the chance to be treated fairly by the lock.

10.5.4. Condition Objects

Often, a thread enters a critical section only to discover that it can't proceed until a condition is fulfilled. Use a *condition object* to manage threads that have acquired a lock but cannot do useful work. In this section, I introduce the implementation of condition objects in the Java library. (For historical reasons, condition objects are often called *condition variables*.)

Let us refine our simulation of the bank. We do not want to transfer money out of an account that does not have the funds to cover the transfer. Note that we cannot use code like

```
if (bank.getBalance(from) >= amount)
  bank.transfer(from, to, amount);
```

It is entirely possible that the current thread will be deactivated between the successful outcome of the test and the call to `transfer`.

```
if (bank.getBalance(from) >= amount)
    // thread might be deactivated at this point
  bank.transfer(from, to, amount);
```

By the time the thread is running again, the account balance may have fallen below the withdrawal amount. You must make sure that no other thread can modify the balance between the test and the transfer action. You do so by protecting both the test and the transfer action with a lock:

```
public void transfer(int from, int to, int amount)
{
  bankLock.lock();
  try
  {
    while (accounts[from] < amount)
    {
      // wait
      . . .
    }
    // transfer funds
    . . .
  }
  finally
  {
```

```
      bankLock.unlock();
    }
  }
```

Now, what do we do when there is not enough money in the account? We wait until some other thread has added funds. But this thread has just gained exclusive access to the `bankLock`, so no other thread has a chance to make a deposit. This is where condition objects come in.

A lock object can have one or more associated condition objects. You obtain a condition object with the `newCondition` method. It is customary to give each condition object a name that evokes the condition that it represents. For example, here we set up a condition object to represent the "sufficient funds" condition.

```
class Bank
{
  . . .
  private final Condition sufficientFunds = bankLock.newCondition();
  . . .
}
```

If the `transfer` method finds that sufficient funds are not available, it calls

```
sufficientFunds.await();
```

The current thread is now deactivated and gives up the lock. This lets in another thread that can, we hope, increase the account balance.

There is an essential difference between a thread that is waiting to acquire a lock and a thread that has called `await`. Once a thread calls the `await` method, it enters a *wait set* for that condition. The thread is *not* made runnable when the lock is available. Instead, it stays deactivated until another thread has called the `signalAll` method on the same condition.

When another thread has transferred money, it should call

```
sufficientFunds.signalAll();
```

This call reactivates all threads waiting for the condition. When the threads are removed from the wait set, they are again runnable and the scheduler will eventually activate them again. At that time, they will attempt to reenter the object. As soon as the lock is available, one of them will acquire the lock *and continue where it left off*, returning from the call to `await`.

At this time, the thread should test the condition again. There is no guarantee that the condition is now fulfilled—the `signalAll` method merely signals to the waiting threads that it *may be* fulfilled at this time and that it is worth checking for the condition again.



Note: In general, a call to `await` should be inside a loop of the form

```
while (!(OK to proceed))
  condition.await();
```

It is crucially important that *some* other thread calls the `signalAll` method eventually. When a thread calls `await`, it has no way of reactivating itself. It puts its faith in the other threads. If none of them bother to reactivate the waiting thread, it will never run again. This can lead to unpleasant *deadlock* situations. If all other threads are blocked and the last active thread calls `await` without unblocking one of the others, it also blocks. No thread is left to unblock the others, and the program hangs.

When should you call `signalAll`? The rule of thumb is to call `signalAll` whenever the state of an object changes in a way that might be advantageous to waiting threads. For example, whenever an account balance changes, the waiting threads should be given another chance to inspect the balance. In our example, we call `signalAll` when we have finished the funds transfer.

```
public void transfer(int from, int to, int amount)
{
  bankLock.lock();
  try
  {
    while (accounts[from] < amount)
      sufficientFunds.await();
    // transfer funds
    . . .
    sufficientFunds.signalAll();
  }
  finally
```

```
    {
      bankLock.unlock();
    }
  }
```

Note that the call to `signalAll` does not immediately activate a waiting thread. It only unblocks the waiting threads so that they can compete for entry into the object after the current thread has relinquished the lock.

Another method, `signal`, unblocks only a single thread from the wait set, chosen at random. That is more efficient than unblocking all threads, but there is a danger. If the randomly chosen thread finds that it still cannot proceed, it becomes blocked again. If no other thread calls `signal` again, the system deadlocks.



Caution: A thread can only call `await`, `signalAll`, or `signal` on a condition if it owns the lock of the condition.

The sample program in the `synch` package has the same `main` method as the `unsynch` version in **Listing 10.5**, but the `Bank` class (**Listing 10.6**) uses locks. You will notice that nothing ever goes wrong. The total balance stays at $100,000 forever. No account ever has a negative balance. (Again, press Ctrl+C to terminate the program.) You may also notice that the program runs a bit slower—that is the price you pay for the added bookkeeping involved in the synchronization mechanism.

When reading through the source code, notice that the `transfer` and `getTotalBalance` methods are protected by a lock, but the `size` method is not. In this simulation, the number of accounts never changes after construction, so there is no need for granting exclusive access to the `size` method.

Listing 10.6 synch/Bank.java

```
1  package synch;
2
3  import java.util.*;
4  import java.util.concurrent.locks.*;
5
6  /**
7   * A bank with a number of bank accounts that uses locks for serializing access.
8   */
9  public class Bank
10 {
```

```
11  private final double[] accounts;
12  private final Lock bankLock = new ReentrantLock();
13  private final Condition sufficientFunds = bankLock.newCondition();
14
15  /**
16   * Constructs the bank.
17   * @param n the number of accounts
18   * @param initialBalance the initial balance for each account
19   */
20  public Bank(int n, double initialBalance)
21  {
22    accounts = new double[n];
23    Arrays.fill(accounts, initialBalance);
24  }
25
26  /**
27   * Transfers money from one account to another.
28   * @param from the account to transfer from
29   * @param to the account to transfer to
30   * @param amount the amount to transfer
31   */
32  public void transfer(int from, int to, double amount) throws InterruptedException
33  {
34    bankLock.lock();
35    try
36    {
37      while (accounts[from] < amount)
38        sufficientFunds.await();
39      System.out.print(Thread.currentThread());
40      accounts[from] -= amount;
41      System.out.printf(" %10.2f from %d to %d", amount, from, to);
42      accounts[to] += amount;
43      System.out.printf(" Total Balance: %10.2f%n", getTotalBalance());
44      sufficientFunds.signalAll();
45    }
46    finally
47    {
48      bankLock.unlock();
49    }
50  }
51
52  /**
53   * Gets the sum of all account balances.
54   * @return the total balance
55   */
```

```
56    public double getTotalBalance()
57    {
58       bankLock.lock();
59       try
60       {
61          double sum = 0;
62
63          for (double a : accounts)
64             sum += a;
65
66          return sum;
67       }
68       finally
69       {
70          bankLock.unlock();
71       }
72    }
73
74    /**
75     * Gets the number of accounts in the bank.
76     * @return the number of accounts
77     */
78    public int size()
79    {
80       return accounts.length;
81    }
82 }
```

java.util.concurrent.locks.Lock **5.0**

- `Condition newCondition()`
  returns a condition object associated with this lock.

java.util.concurrent.locks.Condition **5.0**

- `void await()`
  puts this thread on the wait set for this condition.
- `void signalAll()`
  unblocks all threads in the wait set for this condition.
- `void signal()`
  unblocks one randomly selected thread in the wait set for this condition.

10.5.5. Deadlocks

Locks and conditions cannot solve all problems that might arise in multithreading. Consider the following situation:

1. Account 1: $200
2. Account 2: $300
3. Thread 1: Transfer $300 from Account 1 to Account 2
4. Thread 2: Transfer $400 from Account 2 to Account 1

As **Figure 10.4** indicates, Threads 1 and 2 are clearly blocked. Neither can proceed because the balances in Accounts 1 and 2 are insufficient.

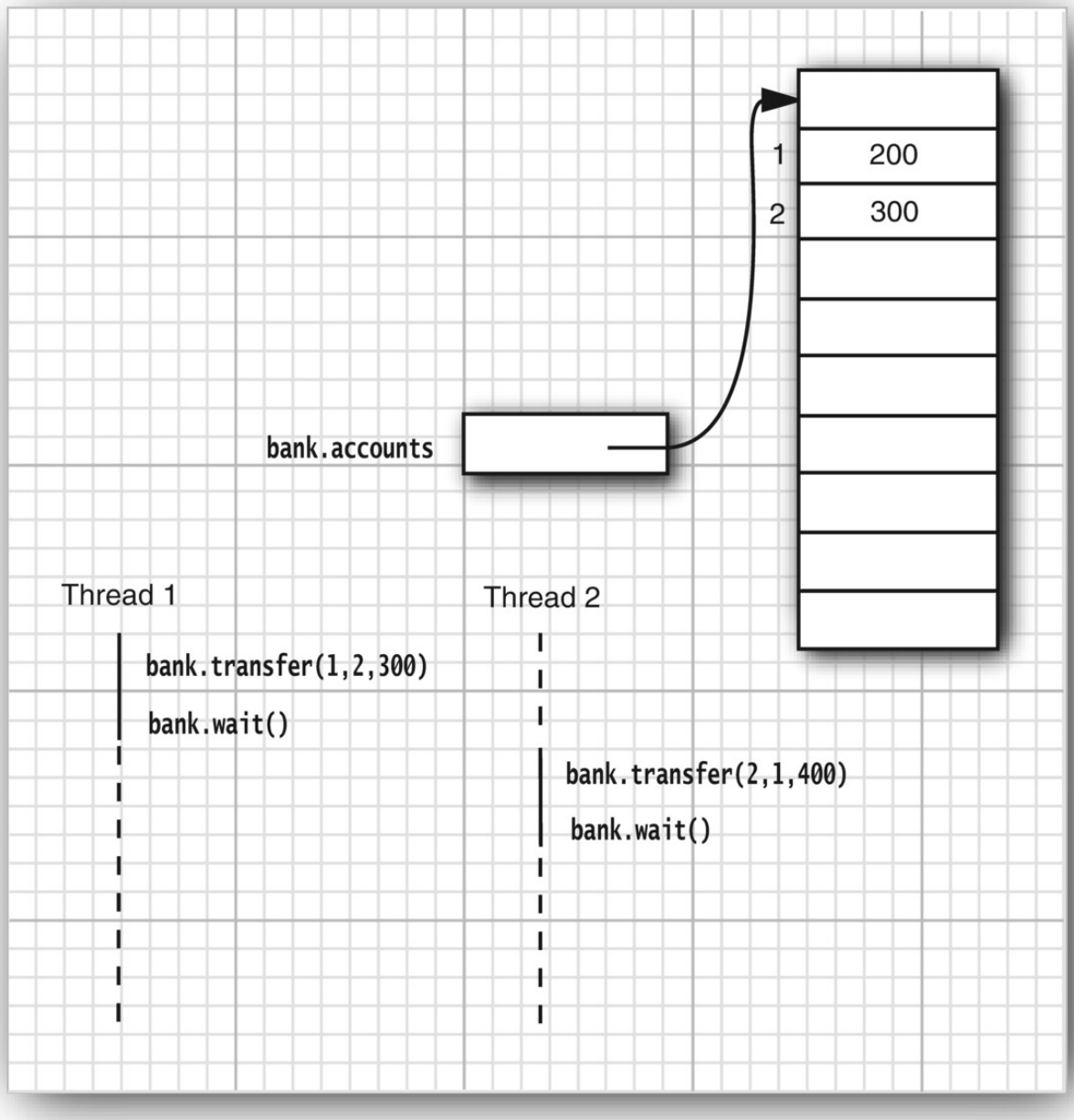


Figure 10.4: A deadlock situation

It is possible that all threads get blocked because each is waiting for more money. Such a situation is called a *deadlock.*

In our program, a deadlock cannot occur for a simple reason. Each transfer amount is for, at most, $1,000. Since there are 100 accounts and a total of $100,000 in them, at least one of the accounts must have at least $1,000 at any time. The thread moving money out of that account can therefore proceed.

But if you change the `run` method of the threads to remove the $1,000 transaction limit, deadlocks will occur quickly. Try it out. Set `NACCOUNTS` to `10`. Construct each transfer runnable with a `max` value of `2 * INITIAL_BALANCE` and run the program. The program will run for a while and then hang.



Tip: When the program hangs, press Ctrl+\. You will get a thread dump that lists all threads. Each thread has a stack trace, telling you where it is currently blocked. Alternatively, run `jconsole`, as described in **Chapter 7**, and consult the Threads panel (see **Figure 10.5**).

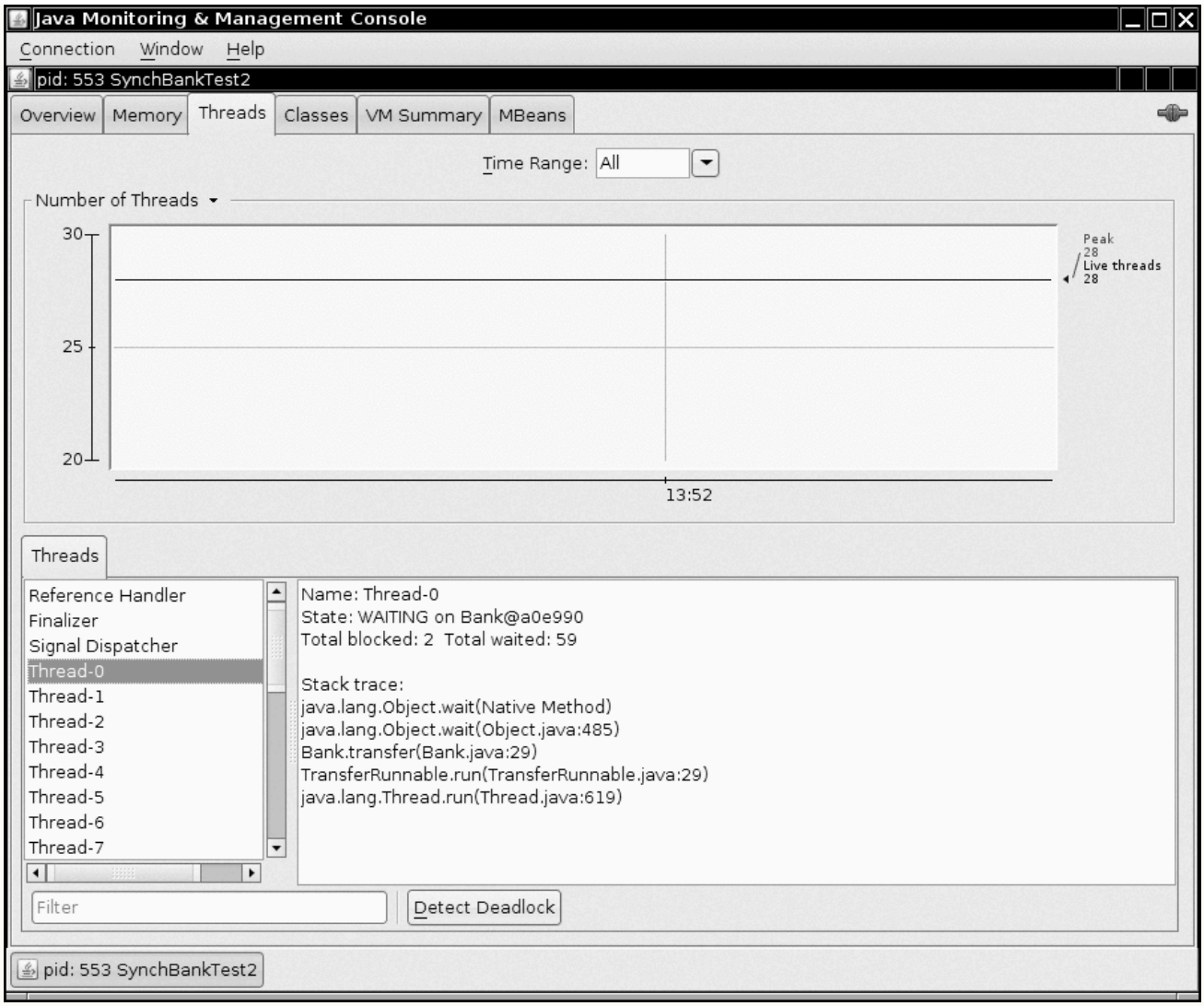


Figure 10.5: The Threads panel in jconsole

Another way to create a deadlock is to make the `i`th thread responsible for putting money into the `i`th account, rather than for taking it out of the `i`th account. In this case, there is a chance that all threads will simultaneously try to overdraw the same account, leaving no thread that can replenish it. Try it out. In the `SynchBankTest` program, turn to the `run` method of the `TransferRunnable` class. In the call to `transfer`, flip `fromAccount` and `toAccount`. Run the program and see how it deadlocks almost immediately.

Here is another situation in which a deadlock can occur easily. Change the `signalAll` method to `signal` in the `SynchBankTest` program. You will find that the program eventually hangs. (Again, set `NACCOUNTS` to `10` to observe the effect more quickly.) Unlike `signalAll`, which notifies all threads that are waiting for added funds, the `signal` method unblocks only one thread. If that thread can't proceed, all threads can be blocked. Consider the following sample scenario of a developing deadlock:

1. Account 1: $1,990
2. All other accounts: $990 each
3. Thread 1: Transfer $995 from Account 1 to Account 2

4. All other threads: Transfer $995 from their account to another account

Clearly, all threads but Thread 1 are blocked, because there isn't enough money in their accounts.

Thread 1 proceeds. Afterward, we have the following situation:

1. Account 1: $995
2. Account 2: $1,985
3. All other accounts: $990 each

Then, Thread 1 calls `signal`. The `signal` method picks a thread at random to unblock. Suppose it picks Thread 3. That thread is awakened, finds that there isn't enough money in its account, and calls `await` again. But Thread 1 is still running. A new random transaction is generated, say,

1. Thread 1: Transfer $997 from Account 1 to Account 2

Now, Thread 1 also calls `await`, and *all* threads are blocked. The system has deadlocked.

The culprit here is the call to `signal`. It only unblocks one thread, and it may not pick the thread that is essential to make progress. (In our scenario, Thread 2 must proceed to take money out of Account 2.)

As you can see, programming with locks and conditions can be quite challenging. You must design your program to ensure that a deadlock situation cannot occur.

10.5.6. The `synchronized` Keyword

In the preceding sections, you saw how to use `Lock` and `Condition` objects. Before going any further, let us summarize the key points about locks and conditions:

- A lock protects sections of code, allowing only one thread to execute the code at a time.
- A lock manages threads that are trying to enter a critical section.
- A lock can have one or more associated condition objects.
- Each condition object manages threads that have entered a critical section but that cannot proceed.

The `Lock` and `Condition` interfaces give programmers a high degree of control over locking. However, in most situations, you don't need that control—you can use a mechanism

that is built into the Java language. Ever since version 1.0, *every object* in Java has an intrinsic lock. If a method is declared with the `synchronized` keyword, the object's lock protects the entire method. That is, to call the method, a thread must acquire the intrinsic object lock.

In other words,

```
public synchronized void method()
{
  method body
}
```

is the equivalent of

```
public void method()
{
  this.intrinsicLock.lock();
  try
  {
    method body
  }
  finally
  {
    this.intrinsicLock.unlock();
  }
}
```

For example, instead of using an explicit lock, we can simply declare the `transfer` method of the `Bank` class as `synchronized`.

The intrinsic object lock has a single associated condition. The `wait` method adds a thread to the wait set, and the `notifyAll` / `notify` methods unblock waiting threads. In other words, calling `wait` or `notifyAll` is the equivalent of

```
intrinsicCondition.await();
intrinsicCondition.signalAll();
```



Note: The `wait`, `notifyAll`, and `notify` methods are `final` methods of the `Object` class. The `Condition` methods had to be named `await`, `signalAll`, and `signal` so

that they don't conflict with those methods.

For example, you can implement the `Bank` class in Java like this:

```
class Bank
{
  private double[] accounts;

  public synchronized void transfer(int from, int to, int amount)
      throws InterruptedException
  {
    while (accounts[from] < amount)
      wait(); // wait on intrinsic object lock's single condition
    accounts[from] -= amount;
    accounts[to] += amount;
    notifyAll(); // notify all threads waiting on the condition
  }

  public synchronized double getTotalBalance()
  {
    . . .
  }
}
```

As you can see, using the `synchronized` keyword yields code that is much more concise. Of course, to understand this code, you have to know that each object has an intrinsic lock, and that the lock has an intrinsic condition. The lock manages the threads that try to enter a `synchronized` method. The condition manages the threads that have called `wait`.

It is also legal to declare static methods as synchronized. If such a method is called, it acquires the intrinsic lock of the associated class object. For example, if the `Bank` class has a static synchronized method, then the lock of the `Bank.class` object is locked when it is called. As a result, no other thread can call this or any other synchronized static method of the same class.

The intrinsic locks and conditions have some limitations. Among them:

- Virtual threads are *pinned* when acquiring an intrinsic lock. They cannot be unmounted from their carrier thread until the lock is relinquished. Pinning is not a problem if the critical section is immediately exited. But if the virtual thread blocks while pinned, this can seriously degrade throughput and even cause starvation of the carrier thread pool.

- You cannot interrupt a thread that is trying to acquire an intrinsic lock. With a `ReentrantLock`, you can call `lockInterruptibly`.
- You cannot specify a timeout when trying to acquire an intrinsic lock.
- Having a single condition per lock can be inefficient.

What should you use in your code—`Lock` and `Condition` objects or synchronized methods? Here is my recommendation:

- It is best to use neither `Lock`/`Condition` nor the `synchronized` keyword. In many situations, you can use one of the mechanisms of the `java.util.concurrent` package that do all the locking for you. For example, in **Section 10.6.1**, you will see how to use a blocking queue to synchronize threads that work on a common task.
- If the `synchronized` keyword works for your situation, by all means, use it. You'll write less code and have less room for error. **Listing 10.7** shows the bank example, implemented with synchronized methods. However, consider the fact that the lock is public—see the next section.
- Use `Lock`/`Condition` if you use virtual threads, or if you really need the additional power that these constructs give you.



Tip: To find out whether pinned threads are blocked, start the JVM with one of the options

```
-Djdk.tracePinnedThreads=short
-Djdk.tracePinnedThreads=full
```

You get a stack trace that shows when a pinned thread blocks:

```
. . .
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49)
  <== monitors:1
. . .
```

Note that you get only one warning per pinning location!

Alternatively, record with Java Flight Recorder, view with your favorite mission control viewer, and look for `VirtualThreadPinned` and `VirtualThreadSubmitFailed` events.

Listing 10.7 synch2/Bank.java

```
1  package synch2;
2
3  import java.util.*;
4
5  /**
6   * A bank with a number of bank accounts that uses synchronization primitives.
7   */
8  public class Bank
9  {
10   private final double[] accounts;
11
12   /**
13    * Constructs the bank.
14    * @param n the number of accounts
15    * @param initialBalance the initial balance for each account
16    */
17   public Bank(int n, double initialBalance)
18   {
19      accounts = new double[n];
20      Arrays.fill(accounts, initialBalance);
21   }
22
23   /**
24    * Transfers money from one account to another.
25    * @param from the account to transfer from
26    * @param to the account to transfer to
27    * @param amount the amount to transfer
28    */
29   public synchronized void transfer(int from, int to, double amount)
30         throws InterruptedException
31   {
32      while (accounts[from] < amount)
33         wait();
34      System.out.print(Thread.currentThread());
35      accounts[from] -= amount;
36      System.out.printf(" %10.2f from %d to %d", amount, from, to);
37      accounts[to] += amount;
38      System.out.printf(" Total Balance: %10.2f%n", getTotalBalance());
39      notifyAll();
40   }
41
42   /**
43    * Gets the sum of all account balances.
```

```
44     * @return the total balance
45     */
46    public synchronized double getTotalBalance()
47    {
48       double sum = 0;
49
50       for (double a : accounts)
51          sum += a;
52
53       return sum;
54    }
55
56    /**
57     * Gets the number of accounts in the bank.
58     * @return the number of accounts
59     */
60    public int size()
61    {
62       return accounts.length;
63    }
64 }
```

java.lang.Object **1.0**

- `void notifyAll()`
  unblocks the threads that called `wait` on this object. This method can only be called from within a synchronized method or block. The method throws an `IllegalMonitorStateException` if the current thread is not the owner of the object's lock.
- `void notify()`
  unblocks one randomly selected thread among the threads that called `wait` on this object. This method can only be called from within a synchronized method or block. The method throws an `IllegalMonitorStateException` if the current thread is not the owner of the object's lock.
- `void wait()`
  causes a thread to wait until it is notified. This method can only be called from within a synchronized method or block. It throws an `IllegalMonitorStateException` if the current thread is not the owner of the object's lock.

- `void wait(long millis)`
- `void wait(long millis, int nanos)`

  cause a thread to wait until it is notified or until the specified amount of time has passed. These methods can only be called from within a synchronized method or block. They throw an `IllegalMonitorStateException` if the current thread is not the owner of the object's lock. The number of nanoseconds may not exceed 1,000,000.

10.5.7. Synchronized Blocks

As we just discussed, every Java object has a lock. A thread can acquire the lock by calling a synchronized method. There is a second mechanism for acquiring the lock: by entering a *synchronized block*. When a thread enters a block of the form

```
synchronized (obj) // this is the syntax for a synchronized block
{
  critical section
}
```

then it acquires the lock for `obj`.

You will sometimes find "ad hoc" locks, such as

```
public class Bank
{
  private double[] accounts;
  private final Object lock = new Object();
  . . .
  public void transfer(int from, int to, int amount)
  {
    synchronized (lock) // an ad-hoc lock
    {
      accounts[from] -= amount;
      accounts[to] += amount;
    }
    System.out.println(. . .);
  }
}
```

Here, the `lock` object is created only to use the lock that every Java object possesses.



Caution: With synchronized blocks, be careful about the lock object. For example, this will not work:

```
private final String lock = "LOCK";
. . .
synchronized (lock) { . . . } // Don't lock on string literal!
```

If this occurs twice in the same program, the locks are *the same object* since string literals are shared. This can lead to a deadlock.

Also, stay away from using primitive type wrappers as locks:

```
private final Integer lock = new Integer(42); // Don't lock on wrappers
```

The constructor call `new Integer(42)` is deprecated, and you don't want a maintenance programmer to change the call to `Integer.valueOf(42)`. If done twice with the same magic number, the lock will be accidentally shared.

If you need to modify a static field, lock on the specific class, not on the value returned by `getClass()`:

```
synchronized (MyClass.class) { staticCounter++; } // OK
synchronized (getClass()) { staticCounter++; } // Don't
```

If the method containing this code is called from a subclass, then `getClass()` returns a different `Class` object! You are no longer guaranteed mutual exclusion!

In general, if you must use `synchronized` blocks, *know your lock object*! You must use the same lock for all critical sections, and nobody else must use your lock.

Sometimes, programmers use the lock of an object to implement additional atomic operations—a practice known as *client-side locking*. Consider, for example, the `Vector` class, which is a list whose methods are synchronized. Now suppose we stored our bank balances in a `Vector<Double>`. Here is a naive implementation of a `transfer` method:

```
public void transfer(Vector<Double> accounts, int from, int to, int amount) // ERROR
{
  accounts.set(from, accounts.get(from) - amount);
```

```
    accounts.set(to, accounts.get(to) + amount);
    System.out.println(. . .);
  }
```

The `get` and `set` methods of the `Vector` class are synchronized, but that doesn't help us. It is entirely possible for a thread to be preempted in the `transfer` method after the first call to `get` has been completed. Another thread may then store a different value into the same position. However, we can hijack the lock:

```
public void transfer(Vector<Double> accounts, int from, int to, int amount)
{
  synchronized (accounts)
  {
    accounts.set(from, accounts.get(from) - amount);
    accounts.set(to, accounts.get(to) + amount);
  }
  System.out.println(. . .);
}
```

This approach works, but it is entirely dependent on the fact that the `Vector` class uses the intrinsic lock for all of its mutator methods. However, is this really a fact? The documentation of the `Vector` class makes no such promise. You have to carefully study the source code and hope that future versions do not introduce unsynchronized mutators. As you can see, client-side locking is very fragile and not generally recommended.



Note: The Java virtual machine has built-in support for synchronized methods. However, synchronized blocks are compiled into a lengthy sequence of bytecodes to manage the intrinsic lock.

10.5.8. The Monitor Concept

Locks and conditions are powerful tools for thread synchronization, but they are not very object-oriented. For many years, researchers have looked for ways to make multithreading safe without forcing programmers to think about explicit locks. One of the most successful solutions is the *monitor* concept that was pioneered by Per Brinch Hansen and Tony Hoare in the 1970s. In the terminology of Java, a monitor has these properties:

- A monitor is a class with only private fields.
- Each object of that class has an associated lock.

- All methods are locked by that lock. In other words, if a client calls `obj.method()`, then the lock for `obj` is automatically acquired at the beginning of the method call and relinquished when the method returns. Since all fields are private, this arrangement ensures that no thread can access the fields while another thread manipulates them.
- The lock can have any number of associated conditions.

Earlier versions of monitors had a single condition, with a rather elegant syntax. You can simply call `await accounts[from] >= amount` without using an explicit condition variable. However, research showed that indiscriminate retesting of conditions can be inefficient. This problem is solved with explicit condition variables, each managing a separate set of threads.

The Java designers loosely adapted the monitor concept. *Every object* in Java has an intrinsic lock and an intrinsic condition. If a method is declared with the `synchronized` keyword, it acts like a monitor method. The condition variable is accessed by calling `wait` / `notifyAll` / `notify`.

However, a Java object differs from a monitor in three important ways, compromising thread safety:

- Fields are not required to be `private`.
- Methods are not required to be `synchronized`.
- The intrinsic lock is available to clients.

This disrespect for security enraged Per Brinch Hansen. In a scathing review of the multithreading primitives in Java, he wrote: "It is astounding to me that Java's insecure parallelism is taken seriously by the programming community, a quarter of a century after the invention of monitors and Concurrent Pascal. It has no merit" (Java's Insecure Parallelism, *ACM SIGPLAN Notices* 34:38–45, April 1999).

10.5.9. Volatile Fields

Sometimes, it seems excessive to pay the cost of synchronization just to read or write an instance field or two. After all, what can go wrong? Unfortunately, with modern processors and compilers, there is plenty of room for error.

- Computers with multiple processors can temporarily hold memory values in registers or local memory caches. As a consequence, threads running in different processors may see different values for the same memory location!

- Compilers can reorder instructions for maximum throughput. Compilers won't choose an ordering that changes the meaning of the code, but they make the assumption that memory values are only changed when there are explicit instructions in the code. However, a memory value can be changed by another thread!

If you use locks to protect code that can be accessed by multiple threads, you won't have these problems. Compilers are required to respect locks by flushing or re-fetching register values, not reordering instructions inappropriately, and issuing memory fence instructions. The details are explained in the Java Memory Model Specification (section 17.4 of the Java Language Specification). Much of the specification is highly complex and technical, but the section also contains a number of clearly explained examples.



Note: Brian Goetz coined the following "synchronization motto": "If you write a variable which may next be read by another thread, or you read a variable which may have last been written by another thread, you must use synchronization."

The `volatile` keyword offers a lock-free mechanism for synchronizing access to an instance field. If you declare a field as `volatile`, then the compiler and the virtual machine take into account that the field may be concurrently updated by another thread.

For example, suppose an object has a `boolean` flag `done` that is set by one thread and queried by another thread. As we already discussed, you can use a lock:

```
private boolean done;
public synchronized boolean isDone() { return done; }
public synchronized void setDone() { done = true; }
```

Perhaps it is not a good idea to use the intrinsic object lock. The `isDone` and `setDone` methods can block if another thread has locked the object. If that is a concern, one can use a separate lock just for this variable. But this is getting to be a lot of trouble.

In this case, it is reasonable to declare the field as `volatile`:

```
private volatile boolean done;
public boolean isDone() { return done; }
public void setDone() { done = true; }
```

The compiler will insert the appropriate code to ensure that a change to the `done` variable in one thread is visible from any other thread that reads the variable.



Caution: Volatile variables do not provide any atomicity. For example, the method

```
public void flipDone() { done = !done; } // not atomic
```

is not guaranteed to flip the value of the field. There is no guarantee that the reading, flipping, and writing is uninterrupted.

10.5.10. Final Fields

As you saw in the preceding section, you cannot safely read a field from multiple threads unless you use locks or the `volatile` modifier.

There is one other situation in which it is safe to access a shared field—when it is declared `final`. Once the constructor has completed, any thread with a reference to the object will see the final fields with their initialized values.

Consider

```
public class Bank
{
  final HashMap<String, Double> accounts;

  public Bank()
  {
    accounts = new HashMap<>();
  }
}
```

Other threads get to see the initialized `accounts` variable after the constructor has finished.

Without using `final`, there would be no guarantee that other threads would see the updated value of `accounts`—they might all see `null`, not the constructed `HashMap`.

Of course, the operations on the map are not thread-safe. If multiple threads mutate and read the map, you still need synchronization.



Caution: The guarantee has an important caveat: the reference to `this` must not have *escaped* during construction. The reference escapes if the constructor assigns

`this` to a field of another object or passes `this` to another method.

An object is *properly constructed* if the `this` reference does not escape during construction. If an object was properly constructed and all fields are `final`, it can be safely shared without synchronization.

10.5.11. Atomics

You can declare shared variables as `volatile` provided you perform no operations other than assignment.

There are a number of classes in the `java.util.concurrent.atomic` package that use efficient machine-level instructions to guarantee atomicity of other operations without using locks. For example, the `AtomicInteger` class has methods `incrementAndGet` and `decrementAndGet` that atomically increment or decrement an integer. For example, you can safely generate a sequence of numbers like this:

```
public static final AtomicLong nextNumber = new AtomicLong();
// in some thread. . .
long id = nextNumber.incrementAndGet();
```

The `incrementAndGet` method atomically increments the `AtomicLong` and returns the post-increment value. That is, the operations of getting the value, adding 1, setting it, and producing the new value cannot be preempted. It is guaranteed that the correct value is computed and returned, even if multiple threads access the same instance concurrently.

There are methods for atomically setting, adding, and subtracting values, but if you want to make a more complex update, you have to use the `compareAndSet` method. For example, suppose you want to keep track of the largest value that is observed by different threads. The following won't work:

```
public static final AtomicLong largest = new AtomicLong();
// in some thread. . .
largest.set(Math.max(largest.get(), observed)); // ERROR--race condition!
```

This update is not atomic. Instead, provide a lambda expression for updating the variable, and the update is done for you. In our example, we can call

```
largest.updateAndGet(x -> Math.max(x, observed));
```

or

```
largest.accumulateAndGet(observed, Math::max);
```

The `accumulateAndGet` method takes a binary operator that is used to combine the atomic value and the supplied argument.

There are also methods `getAndUpdate` and `getAndAccumulate` that return the old value.



Note: These methods are also provided for the classes `AtomicInteger`, `AtomicIntegerArray`, `AtomicIntegerFieldUpdater`, `AtomicLongArray`, `AtomicLongFieldUpdater`, `AtomicReference`, `AtomicReferenceArray`, and `AtomicReferenceFieldUpdater`.

When you have a very large number of threads accessing the same atomic values, performance suffers because the optimistic updates require too many retries. The `LongAdder` and `LongAccumulator` classes solve this problem. A `LongAdder` is composed of multiple variables whose collective sum is the current value. Multiple threads can update different summands, and new summands are automatically provided when the number of threads increases. This is efficient in the common situation where the value of the sum is not needed until after all work has been done. The performance improvement can be substantial.

If you anticipate high contention, you should simply use a `LongAdder` instead of an `AtomicLong`. The method names are slightly different. Call `increment` to increment a counter or `add` to add a quantity, and `sum` to retrieve the total.

```
final var adder = new LongAdder();
for (. . .)
  executor.submit(() ->
    {
      while (. . .)
      {
        . . .
        if (. . .) adder.increment();
      }
```

```
    });
. . .
long total = adder.sum();
```



Note: Of course, the `increment` method does *not* return the old value. Doing that would undo the efficiency gain of splitting the sum into multiple summands.

The `LongAccumulator` generalizes this idea to an arbitrary accumulation operation. In the constructor, you provide the operation, as well as its neutral element. To incorporate new values, call `accumulate`. Call `get` to obtain the current value. The following has the same effect as a `LongAdder`:

```
final var adder = new LongAccumulator(Long::sum, 0);
// in some thread. . .
adder.accumulate(value);
```

Internally, the accumulator has variables a_1, a_2, . . ., a_n. Each variable is initialized with the neutral element (`0` in our example).

When `accumulate` is called with value v, then one of them is atomically updated as $a_i = a_i$ op v, where op is the accumulation operation written in infix form. In our example, a call to `accumulate` computes $a_i = a_i + v$ for some i.

The result of `get` is a_1 op a_2 op . . . op a_n. In our example, that is the sum of the accumulators, $a_1 + a_2 + . . . + a_n$.

To compute maximum or minimum, use `Math.max` or `Math.min` for the operation. In general, the operation must be associative and commutative. That means that the final result must be independent of the order in which the intermediate values were combined.

There are also `DoubleAdder` and `DoubleAccumulator` that work in the same way, except with `double` values.

10.5.12. On-Demand Initialization

Sometimes, you have a data structure that you only want to initialize when it is first needed. And you want to ensure that initialization happens exactly once. Instead of designing your own mechanism, make use of the fact that the virtual machine executes a

static initializer exactly once when the class is first used. The virtual machine ensures this with a lock, so you don't have to program your own.

```
public class OnDemandData
{
  // private constructor to ensure only one object is constructed
  private OnDemandData()
  {
    . . .
  }

  public static OnDemandData getInstance()
  {
    return Holder.INSTANCE;
  }

  // only initialized on first use, i.e. in the first call to getInstance
  private static class Holder
  {
    // VM guarantees that this happens lazily and at most once
    static final OnDemandData INSTANCE = new OnDemandData();
  }
}
```



Caution: To use this idiom, you must ensure that the constructor doesn't throw any exceptions. The virtual machine will not make a second attempt to initialize the holder class.

10.5.13. Safe Publication

As you saw in **Section 10.5.10**, the `final` fields of a properly constructed object will be seen by all threads with their initialized values. What about the non-final fields?

The Java language makes no guarantee in general. For example, consider:

```
public class BankAccount
{
  private double balance;

  public BankAccount(double initialBalance) { balance = initialBalance; }
```

```
    public double getBalance() { return balance; }
    . . .
}
```

If you construct a `new BankAccount(1000)` object in one thread, and another thread has a reference to the object, it might see `getBalance()` return zero.

Of course, a mutable `BankAccount` class should have thread-safe methods `getBalance`, `deposit`, `withdraw` that protect the `balance` field with a lock. Then `getBalance` will report the correct balance.

Now consider the hopefully uncommon case where you share an object that is neither thread-safe nor immutable. Then you need to ensure that the object is *safely published*—that is, all fields are visible with the values that were set in the constructor. This is ensured when the object reference is stored:

- In a static initializer (as in the preceding section)
- In a `volatile` field or `AtomicReference`
- In any field when the assignment is protected by a lock
- In a `final` field of a properly constructed object

The third condition is fulfilled when a `Runnable` or `Callable` is an inner class or lambda expression that captures a variable from the enclosing scope. The captured variable is stored in a `final` field of the inner class.

The last condition is fulfilled when you store the object reference in a thread-safe data structure, for example a `BlockingQueue` or `ConcurrentHashMap`. The retrieving thread will see the object in its published state.

As you can see, in most reasonable circumstances, nothing evil will happen. If you find yourself in a situation where these details matter, you are on thin ice and may want to rethink your sharing strategy.

10.5.14. Sharing with Thread-Local Variables

In the preceding sections, we discussed the risks of sharing variables between threads. Sometimes, you can avoid sharing by giving each thread its own instance, using the thread-local variables (see **Section 10.4.4**). For example, the `SimpleDateFormat` class is not thread-safe. Suppose we have a static variable

```
public static final SimpleDateFormat DATEFORMAT = new SimpleDateFormat("yyyy-MM-dd");
```

If two threads execute an operation such as

```
String DATEFORMAT = dateFormat.format(new Date());
```

then the result can be garbage since the internal data structures used by the `dateFormat` can be corrupted by concurrent access. You could use synchronization, which is expensive, or you could construct a local `SimpleDateFormat` object whenever you need it, but that is also wasteful.

To construct one instance per thread, use the following code:

```
public static final ThreadLocal<SimpleDateFormat> DATEFORMAT =
  ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
```

To access the actual formatter, call

```
String dateStamp = DATEFORMAT.get().format(new Date());
```

The first time you call `get` in a given thread, the lambda in the constructor is called. From then on, the `get` method returns the instance belonging to the current thread.



Preview Note: As pointed out in Section 10.4.4, thread-local variables may not be appropriate for large numbers of virtual threads. Extent locals, a preview feature in Java 21, provide a more performant alternative.

A related problem is the generation of random numbers in multiple threads. The `java.util.Random` class is thread-safe. However, the computation that yields a new random value and updates the state of the generator can be inefficient if multiple threads use a single shared generator.

You could use the `ThreadLocal` helper to give each thread a separate generator. But there is a convenience class that is optimized for this situation:

```
int random = ThreadLocalRandom.current().nextInt(upperBound);
```

The call `ThreadLocalRandom.current()` returns a random number generator with a per-thread state. In its current implementation, it stores the generator state in instance variables of the `Thread` object.

`java.lang.ThreadLocal<T>` **1.2**

- `static <S> ThreadLocal<S> withInitial(Supplier<? extends S> supplier)` **8**
  creates a thread-local variable whose initial value is produced by invoking the given supplier.

`java.util.concurrent.ThreadLocalRandom` **7**

- `static ThreadLocalRandom current()`
  returns a random generator class whose state is local to the current thread.

10.6. Thread-Safe Collections

If multiple threads concurrently modify a data structure, such as a hash table, it is easy to damage that data structure. (See **Chapter 9** for more information on hash tables.) For example, one thread may begin to insert a new element. Suppose it is preempted in the middle of rerouting the links between the hash table's buckets. If another thread starts traversing the same list, it may follow invalid links and create havoc, perhaps throwing exceptions or getting trapped in an infinite loop.

You can protect a shared data structure by supplying a lock, but it is usually easier to choose a thread-safe implementation instead. In the following sections, we discuss the thread-safe collections that the Java library provides.

10.6.1. Blocking Queues

Many threading problems can be formulated elegantly and safely by using one or more queues. Producer threads insert items into the queue, and consumer threads retrieve them. The queue lets you safely hand over data from one thread to another. For example, consider our bank transfer program. Instead of accessing the bank object directly, the transfer threads insert transfer instruction objects into a queue. Another thread removes the instructions from the queue and carries out the transfers. Only that thread has access to the internals of the bank object. No synchronization is necessary. (Of course, the implementors of the thread-safe queue classes had to worry about locks and conditions, but that was their problem, not yours.)

A *blocking queue* causes a thread to block when you try to add an element when the queue is currently full or to remove an element when the queue is empty. Blocking queues are a useful tool for coordinating the work of multiple threads. Worker threads can periodically deposit intermediate results into a blocking queue. Other worker threads remove the intermediate results and modify them further. The queue automatically balances the workload. If the first set of threads runs slower than the second, the second set blocks while waiting for the results. If the first set of threads runs faster, the queue fills up until the second set catches up. **Table 10.2** shows the methods for blocking queues.

Table 10.2: Blocking Queue Methods

Method	Normal Action	Action in Special Circumstances
`add`	Adds an element	Throws an `IllegalStateException` if the queue is full
`element`	Returns the head element	Throws a `NoSuchElementException` if the queue is empty
`offer`	Adds an element and returns `true`	Returns `false` if the queue is full
`peek`	Returns the head element	Returns `null` if the queue is empty
`poll`	Removes and returns the head element	Returns `null` if the queue is empty
`put`	Adds an element	Blocks if the queue is full
`remove`	Removes and returns the head element	Throws a `NoSuchElementException` if the queue is empty
`take`	Removes and returns the head element	Blocks if the queue is empty

The blocking queue methods fall into three categories that differ by the action they perform when the queue is full or empty. If you use the queue as a thread management tool, use the `put` and `take` methods. The `add`, `remove`, and `element` operations throw an exception when you try to add to a full queue or get the head of an empty queue. Of

course, in a multithreaded program, the queue might become full or empty at any time, so you will instead want to use the `offer`, `poll`, and `peek` methods. These methods simply return with a failure indicator instead of throwing an exception if they cannot carry out their tasks.



Note: The `poll` and `peek` methods return `null` to indicate failure. Therefore, it is illegal to insert `null` values into these queues.

There are also variants of the `offer` and `poll` methods with a timeout. For example, the call

```
boolean success = q.offer(x, 100, TimeUnit.MILLISECONDS);
```

tries for 100 milliseconds to insert an element to the tail of the queue. If it succeeds, it returns `true`; otherwise, it returns `false` when it times out. Similarly, the call

```
Object head = q.poll(100, TimeUnit.MILLISECONDS);
```

tries for 100 milliseconds to remove the head of the queue. If it succeeds, it returns the head; otherwise, it returns `null` when it times out.

The `put` method blocks if the queue is full, and the `take` method blocks if the queue is empty. These are the equivalents of `offer` and `poll` with no timeout.

The `java.util.concurrent` package supplies several variations of blocking queues. By default, the `LinkedBlockingQueue` has no upper bound on its capacity, but a maximum capacity can be optionally specified. The `LinkedBlockingDeque` is a double-ended version. The `ArrayBlockingQueue` is constructed with a given capacity and an optional parameter to require fairness. If fairness is specified, then the longest-waiting threads are given preferential treatment. As always, fairness exacts a significant performance penalty, and you should only use it if your problem specifically requires it.

The `PriorityBlockingQueue` is a priority queue, not a first-in/first-out queue. Elements are removed in order of their priority. The queue has unbounded capacity, but retrieval will block if the queue is empty. (See **Chapter 9** for more information on priority queues.)

The `TransferQueue` interface allows a producer thread to wait until a consumer is ready to take on an item. When a producer calls

```
q.transfer(item);
```

the call blocks until another thread removes it. The `LinkedTransferQueue` class implements this interface.

The program in **Listing 10.8** shows how to use a blocking queue to control a set of threads. The program searches through all files in a directory and its subdirectories, printing lines that contain a given keyword.

A producer thread enumerates all files in all subdirectories and places them in a blocking queue. This operation is fast, and the queue would quickly fill up with all files in the file system if it was not bounded.

We also start a large number of search threads. Each search thread takes a file from the queue, opens it, prints all lines containing the keyword, and then takes the next file. We use a trick to terminate the application when no further work is required. In order to signal completion, the enumeration thread places a special termination object into the queue. (This is similar to a dummy suitcase with a label "last bag" in a baggage claim belt.) When a search thread takes the special object, it terminates, after putting the object back for other consumers to see.



Note: Some programmers use the term "poison pill" for an object that signals termination to the consumers.

Note that no explicit thread synchronization is required. In this application, we use the queue data structure as a synchronization mechanism.

Listing 10.8 blockingQueue/BlockingQueueTest.java

```
1  package blockingQueue;
2
3  import java.io.*;
4  import java.nio.file.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.stream.*;
8
9  /**
10  * @version 1.04 2023-07-30
11  * @author Cay Horstmann
```

```
 */
public class BlockingQueueTest
{
  private static final int FILE_QUEUE_SIZE = 10;
  private static final int SEARCH_THREADS = 100;
  private static final Path TERMINATION = Path.of("");
  private static BlockingQueue<Path> queue = new ArrayBlockingQueue<>(FILE_QUEUE_SIZE);

  public static void main(String[] args)
  {
    try (var in = new Scanner(System.in))
    {
      System.out.print("Enter base directory (e.g. /tmp/jdk-21-src): ");
      String directory = in.nextLine();
      System.out.print("Enter keyword (e.g. volatile): ");
      String keyword = in.nextLine();

      Runnable enumerator = () ->
        {
          try
          {
            enumerate(Path.of(directory));
            queue.put(TERMINATION);
          }
          catch (IOException e)
          {
            e.printStackTrace();
          }
          catch (InterruptedException e)
          {
          }
        };

      Thread.ofPlatform().start(enumerator);
      for (int i = 1; i <= SEARCH_THREADS; i++)
      {
        Runnable searcher = () ->
          {
            try
            {
              boolean done = false;
              while (!done)
              {
                Path file = queue.take();
                if (file == TERMINATION)
```

```
57                {
58                  queue.put(file);
59                  done = true;
60                }
61                else search(file, keyword);
62              }
63            }
64            catch (IOException e)
65            {
66              e.printStackTrace();
67            }
68            catch (InterruptedException e)
69            {
70            }
71          };
72        Thread.ofPlatform().start(searcher);
73      }
74    }
75  }
76
77  /**
78   * Recursively enumerates all files in a given directory and its subdirectories.
79   * See Chapters 1 and 2 of Volume II for the stream and file operations.
80   * @param directory the directory in which to start
81   */
82  public static void enumerate(Path directory) throws IOException, InterruptedException
83  {
84    try (Stream<Path> children = Files.list(directory))
85    {
86      for (Path child : children.toList())
87      {
88        if (Files.isDirectory(child))
89          enumerate(child);
90        else
91          queue.put(child);
92      }
93    }
94  }
95
96  /**
97   * Searches a file for a given keyword and prints all matching lines.
98   * @param file the file to search
99   * @param keyword the keyword to search for
100  */
101 public static void search(Path file, String keyword) throws IOException
```

```
102   {
103     try (var in = new Scanner(file))
104     {
105       int lineNumber = 0;
106       while (in.hasNextLine())
107       {
108         lineNumber++;
109         String line = in.nextLine();
110         if (line.contains(keyword))
111           System.out.printf("%s:%d:%s%n", file, lineNumber, line);
112       }
113     }
114   }
115 }
```

java.util.concurrent.ArrayBlockingQueue<E> **5.0**

- `ArrayBlockingQueue(int capacity)`
- `ArrayBlockingQueue(int capacity, boolean fair)`

  construct a blocking queue with the given capacity and fairness settings. The queue is implemented as a circular array.

java.util.concurrent.LinkedBlockingQueue<E> **5.0**

java.util.concurrent.LinkedBlockingDeque<E> **6**

- `LinkedBlockingQueue()`
- `LinkedBlockingDeque()`

  construct an unbounded blocking queue or deque, implemented as a linked list.
- `LinkedBlockingQueue(int capacity)`
- `LinkedBlockingDeque(int capacity)`

  construct a bounded blocking queue or deque with the given capacity, implemented as a linked list.

java.util.concurrent.PriorityBlockingQueue<E> 5.0

- PriorityBlockingQueue()
- PriorityBlockingQueue(int initialCapacity)
- PriorityBlockingQueue(int initialCapacity, Comparator<? super E> comparator)

construct an unbounded blocking priority queue implemented as a heap. The default for the initial capacity is 11. If the comparator is not specified, the elements must implement the Comparable interface.

java.util.concurrent.BlockingQueue<E> 5.0

- void put(E element)

adds the element, blocking if necessary.

- E take()

removes and returns the head element, blocking if necessary.

- boolean offer(E element, long time, TimeUnit unit)

adds the given element and returns true if successful, blocking if necessary until the element has been added or the time has elapsed.

- E poll(long time, TimeUnit unit)

removes and returns the head element, blocking if necessary until an element is available or the time has elapsed. Returns null upon failure.

java.util.concurrent.BlockingDeque<E> 6

- void putFirst(E element)
- void putLast(E element)

add the element, blocking if necessary.

- E takeFirst()
- E takeLast()

remove and return the head or tail element, blocking if necessary.

- boolean offerFirst(E element, long time, TimeUnit unit)
- boolean offerLast(E element, long time, TimeUnit unit)

add the given element and return true if successful, blocking if necessary until the element has been added or the time has elapsed.

- E pollFirst(long time, TimeUnit unit)
- E pollLast(long time, TimeUnit unit)

remove and return the head or tail element, blocking if necessary until an element is available or the time has elapsed. Returns null upon failure.

java.util.concurrent.TransferQueue<E> 7

- void transfer(E element)
- boolean tryTransfer(E element, long time, TimeUnit unit)

transfer a value, or try transferring it with a given timeout, blocking until another thread has removed the item. The second method returns true if successful.

10.6.2. Efficient Maps, Sets, and Queues

The java.util.concurrent package supplies efficient implementations for maps, sorted sets, and queues: ConcurrentHashMap, ConcurrentSkipListMap, ConcurrentSkipListSet, and ConcurrentLinkedQueue.

These collections use sophisticated algorithms that minimize contention by allowing concurrent access to different parts of the data structure.

Unlike most collections, the size method of these classes does not necessarily operate in constant time. Determining the current size of one of these collections usually requires traversal.



Note: Some applications use humongous concurrent hash maps, so large that the size method is insufficient because it returns an int. What is one to do with a map that has over two billion entries? The mappingCount method returns the size as a long.

The collections return *weakly consistent* iterators. That means that the iterators may or may not reflect all modifications that are made after they were constructed, but they will not return a value twice and they will not throw a ConcurrentModificationException.



Note: In contrast, an iterator of a collection in the java.util package throws a ConcurrentModificationException when the collection has been modified after construction of the iterator.

The concurrent hash map can efficiently support a large number of readers and a bounded number of writers.



Note: A hash map keeps all entries with the same hash code in the same "bucket." Some applications use poor hash functions, and as a result all entries end up in a small number of buckets, severely degrading performance. Even generally reason-

able hash functions, such as that of the `String` class, can be problematic. For example, an attacker can slow down a program by crafting a large number of strings that hash to the same value. In recent Java versions, the concurrent hash map organizes the buckets as trees, not lists, when the key type implements `Comparable`, guaranteeing O(log n) performance.

`java.util.concurrent.ConcurrentLinkedQueue<E>` **5.0**

- `ConcurrentLinkedQueue<E>()`
  constructs an unbounded, nonblocking queue that can be safely accessed by multiple threads.

`java.util.concurrent.ConcurrentHashMap<K, V>` **5.0**

- `ConcurrentHashMap<K, V>()`
- `ConcurrentHashMap<K, V>(int initialCapacity)`
- `ConcurrentHashMap<K, V>(int initialCapacity, float loadFactor, int concurrencyLevel)`
  construct a hash map that can be safely accessed by multiple threads. The default for the initial capacity is `16`. If the average load per bucket exceeds the load factor, the table is resized. The default is `0.75`. The concurrency level is the estimated number of concurrent writer threads.

`java.util.concurrent.ConcurrentSkipListSet<E>` **6**

- `ConcurrentSkipListSet<E>()`
- `ConcurrentSkipListSet<E>(Comparator<? super E> comp)`
  construct a sorted set that can be safely accessed by multiple threads. The first constructor requires that the elements implement the `Comparable` interface.

`java.util.concurrent.ConcurrentSkipListMap<K, V>` **6**

- `ConcurrentSkipListMap<K, V>()`
- `ConcurrentSkipListMap<K, V>(Comparator<? super K> comp)`
  construct a sorted map that can be safely accessed by multiple threads. The first constructor requires that the keys implement the `Comparable` interface.

10.6.3. Atomic Update of Map Entries

The original version of `ConcurrentHashMap` only had a few methods for atomic updates, which made for somewhat awkward programming. Suppose we want to count how often certain features are observed. As a simple example, suppose multiple threads encounter words, and we want to count their frequencies.

Can we use a `ConcurrentHashMap<String, Long>`? Consider the code for incrementing a count. Obviously, the following is not thread-safe:

```
Long oldValue = map.get(word);
Long newValue = oldValue == null ? 1 : oldValue + 1;
map.put(word, newValue); // ERROR--might not replace oldValue
```

Another thread might be updating the exact same count at the same time.



Note: Some programmers are surprised that a supposedly thread-safe data structure permits operations that are not thread-safe. But there are two entirely different considerations. If multiple threads modify a plain `HashMap`, they can destroy the internal structure (an array of linked lists). Some of the links may go missing, or even go in circles, rendering the data structure unusable. That will never happen with a `ConcurrentHashMap`. In the example above, the code for `get` and `put` will never corrupt the data structure. But, since the sequence of operations is not atomic, the result is not predictable.

In old versions of Java, it was necessary to use the `replace` method, which atomically replaces an old value with a new one, provided that no other thread has come before and replaced the old value with something else. You had to keep doing it until the attempt succeeded:

```
do
{
  oldValue = map.get(word);
  newValue = oldValue == null ? 1 : oldValue + 1;
}
while (!map.replace(word, oldValue, newValue));
```

An alternative was to use a `ConcurrentHashMap<String, AtomicLong>` and the following update code:

```
map.putIfAbsent(word, new AtomicLong());
map.get(word).incrementAndGet();
```

Unfortunately, in the first line, a new `AtomicLong` is constructed for each increment, and it will be discarded when it is not needed.

Nowadays, the Java API provides methods that make atomic updates more convenient and performant. The `compute` method is called with a key and a function to compute the new value. That function receives the key and the associated value, or `null` if there is none, and it computes the new value. For example, here is how we can update a map of integer counters:

```
map.compute(word, (k, v) -> v == null ? 1 : v + 1);
```



Note: You cannot have `null` values in a `ConcurrentHashMap`. There are many methods that use a `null` value as an indication that a given key is not present in the map.

There are also variants `computeIfPresent` and `computeIfAbsent` that only compute a new value when there is already an old one, or when there isn't yet one. A map of `LongAdder` counters can be updated with

```
map.computeIfAbsent(word, k -> new LongAdder()).increment();
```

That is almost like the call to `putIfAbsent` that you saw before, but the `LongAdder` constructor is only called when a new counter is actually needed.

You often need to do something special when a key is added for the first time. The `merge` method makes this particularly convenient. It has a parameter for the initial value that is used when the key is not yet present. Otherwise, the function that you supplied is called, combining the existing value and the initial value. (Unlike `compute`, the function does *not* process the key.)

```
map.merge(word, 1L, (existingValue, newValue) -> existingValue + newValue);
```

or simply

```
map.merge(word, 1L, Long::sum);
```

It doesn't get more concise than that.



Note: If the function that is passed to `compute` or `merge` returns `null`, the existing entry is removed from the map.



Caution: When you use `compute` or `merge`, keep in mind that the function that you supply should not do a lot of work. While that function runs, some other updates to the map may be blocked. Of course, that function should also not update other parts of the map.

The program in **Listing 10.9** uses a concurrent hash map to count all words in the Java files of a directory tree.

Listing 10.9 concurrentHashMap/CHMDemo.java

```
1  package concurrentHashMap;
2
3  import java.io.*;
4  import java.nio.file.*;
5  import java.util.*;
6  import java.util.concurrent.*;
7  import java.util.stream.*;
8
9  /**
10  * This program demonstrates concurrent hash maps.
11  * @version 1.01 2023-07-30
12  * @author Cay Horstmann
13  */
14 public class CHMDemo
15 {
16    public static ConcurrentHashMap<String, Long> map = new ConcurrentHashMap<>();
17
18    /**
19     * Adds all words in the given file to the concurrent hash map.
20     * @param file a file
21     */
22    public static void process(Path file)
23    {
24       try (var in = new Scanner(file))
25       {
```

```
26          while (in.hasNext())
27          {
28             String word = in.next();
29             map.merge(word, 1L, Long::sum);
30          }
31       }
32       catch (IOException e)
33       {
34          e.printStackTrace();
35       }
36    }
37
38    /**
39     * Returns all descendants of a given directory--see Chapters 1 and 2 of Volume II
40     * @param rootDir the root directory
41     * @return a set of all descendants of the root directory
42     */
43    public static Set<Path> descendants(Path rootDir) throws IOException
44    {
45       try (Stream<Path> entries = Files.walk(rootDir))
46       {
47          return entries.collect(Collectors.toSet());
48       }
49    }
50
51    public static void main(String[] args)
52          throws InterruptedException, ExecutionException, IOException
53    {
54       ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
55       Path pathToRoot = Path.of(".");
56       for (Path p : descendants(pathToRoot))
57       {
58          if (p.getFileName().toString().endsWith(".java"))
59             executor.execute(() -> process(p));
60       }
61       executor.close();
62       map.forEach((k, v) ->
63          {
64             if (v >= 10)
65                System.out.println(k + " occurs " + v + " times");
66          });
67    }
68 }
```

10.6.4. Bulk Operations on Concurrent Hash Maps

The Java API provides bulk operations on concurrent hash maps that can safely execute even while other threads operate on the map. The bulk operations traverse the map and operate on the elements they find as they go along. No effort is made to freeze a snapshot of the map in time. Unless you happen to know that the map is not being modified while a bulk operation runs, you should treat its result as an approximation of the map's state.

There are three kinds of operations:

- `search` applies a function to each key and/or value, until the function yields a non-`null` result. Then the search terminates and the function's result is returned.
- `reduce` combines all keys and/or values, using a provided accumulation function.
- `forEach` applies a function to all keys and/or values.

Each operation has four versions:

- `operationKeys` : operates on keys.
- `operationValues` : operates on values.
- `operation` : operates on keys and values.
- `operationEntries` : operates on `Map.Entry` objects.

With each of the operations, you need to specify a *parallelism threshold*. If the map contains more elements than the threshold, the bulk operation is parallelized. If you want the bulk operation to run in a single thread, use a threshold of `Long.MAX_VALUE`. If you want the maximum number of threads to be made available for the bulk operation, use a threshold of `1`.

Let's look at the `search` methods first. Here are the versions:

```
U searchKeys(long threshold, Function<? super K, ? extends U> f)
U searchValues(long threshold, Function<? super V, ? extends U> f)
U search(long threshold, BiFunction<? super K, ? super V,? extends U> f)
U searchEntries(long threshold, Function<Map.Entry<K, V>, ? extends U> f)
```

For example, suppose we want to find the first word that occurs more than 1,000 times. We need to search keys and values:

```
String result = map.search(threshold, (k, v) -> v > 1000 ? k : null);
```

Then `result` is set to the first match, or to `null` if the search function returns `null` for all inputs.

The `forEach` methods have two variants. The first one simply applies a *consumer* function for each map entry, for example

```
map.forEach(threshold,
  (k, v) -> System.out.println(k + " -> " + v));
```

The second variant takes an additional *transformer* function, which is applied first, and its result is passed to the consumer:

```
map.forEach(threshold,
  (k, v) -> k + " -> " + v, // transformer
  System.out::println); // consumer
```

The transformer can be used as a filter. Whenever the transformer returns `null`, the value is silently skipped. For example, here we only print the entries with large values:

```
map.forEach(threshold,
  (k, v) -> v > 1000 ? k + " -> " + v : null, // filter and transformer
  System.out::println); // the nulls are not passed to the consumer
```

The `reduce` operations combine their inputs with an accumulation function. For example, here is how you can compute the sum of all values:

```
Long sum = map.reduceValues(threshold, Long::sum);
```

As with `forEach`, you can also supply a transformer function. Here we compute the length of the longest key:

```
Integer maxlength = map.reduceKeys(threshold,
  String::length, // transformer
  Integer::max); // accumulator
```

The transformer can act as a filter, by returning `null` to exclude unwanted inputs. Here, we count how many entries have value > 1000:

```
Long count = map.reduceValues(threshold,
  v -> v > 1000 ? 1L : null,
  Long::sum);
```



Note: If the map is empty, or all entries have been filtered out, the `reduce` operation returns `null`. If there is only one element, its transformation is returned, and the accumulator is not applied.

There are specializations for `int`, `long`, and `double` outputs with suffixes `ToInt`, `ToLong`, and `ToDouble`. You need to transform the input to a primitive value and specify a default value and an accumulator function. The default value is returned when the map is empty.

```
long sum = map.reduceValuesToLong(threshold,
  Long::longValue, // transformer to primitive type
  0, // default value for empty map
  Long::sum); // primitive type accumulator
```



Caution: These specializations act differently from the object versions where there is only one element to be considered. Instead of returning the transformed element, it is accumulated with the default. Therefore, the default must be the neutral element of the accumulator.

10.6.5. Concurrent Set Views

Suppose you want a large, thread-safe set instead of a map. There is no `ConcurrentHashSet` class, and you know better than trying to create your own. Of course, you can use a `ConcurrentHashMap` with bogus values, but then you get a map, not a set, and you can't apply operations of the `Set` interface.

The static `newKeySet` method yields a `Set<K>` that is actually a wrapper around a `ConcurrentHashMap<K, Boolean>`. (All map values are `Boolean.TRUE`, but you don't actually care since you just use it as a set.)

```
Set<String> words = ConcurrentHashMap.<String>newKeySet();
```

Of course, if you have an existing map, the `keySet` method yields the set of keys. That set is mutable. If you remove the set's elements, the keys (and their values) are removed from the map. But it doesn't make sense to add elements to the key set, because there would be no corresponding values to add. There is a second `keySet` method to `ConcurrentHashMap`, with a default value, to be used when adding elements to the set:

```
Set<String> words = map.keySet(1L);
words.add("Java");
```

If `"Java"` wasn't already present in `words`, it now has a value of one.



Note: You could also use the `Collections.newSetFromMap` method, but `newKeySet` is simpler and a bit more efficient.

10.6.6. Copy on Write Arrays

The `CopyOnWriteArrayList` and `CopyOnWriteArraySet` are thread-safe collections in which all mutators make a copy of the underlying array. This arrangement is useful if the threads that iterate over the collection greatly outnumber the threads that mutate it. When you construct an iterator, it contains a reference to the current array. If the array is later mutated, the iterator still has the old array, but the collection's array is replaced. As a consequence, the older iterator has a consistent (but potentially outdated) view that it can access without any synchronization expense.

10.6.7. Parallel Array Algorithms

The `Arrays` class has a number of parallelized operations. The static `Arrays.parallelSort` method can sort an array of primitive values or objects. For example,

```
var contents = Files.readString(Path.of("alice.txt"));
String[] words = contents.split("\\PL+"); // split along nonletters
Arrays.parallelSort(words);
```

When you sort objects, you can supply a `Comparator`.

```
Arrays.parallelSort(words, Comparator.comparing(String::length));
```

With all methods, you can supply the bounds of a range, such as

```
Arrays.parallelSort(words, words.length / 2, words.length); // sort the upper half
```



Note: At first glance, it seems a bit odd that these methods have `parallel` in their name, since the user shouldn't care how the sorting happens. However, the API designers wanted to make it clear that the sorting is parallelized. That way, users are on notice to avoid comparators with side effects.

The `parallelSetAll` method fills an array with values that are computed from a function. The function receives the element index and computes the value at that location.

```
Arrays.parallelSetAll(values, i -> i % 10);
  // fills values with 0 1 2 3 4 5 6 7 8 9 0 1 2 . . .
```

Clearly, this operation benefits from being parallelized. There are versions for all primitive type arrays and for object arrays.

Finally, there is a `parallelPrefix` method that replaces each array element with the accumulation of the prefix for a given associative operation. Huh? Here is an example. Consider the array `[1, 2, 3, 4, . . .]` and the × operation. After executing `Arrays.parallelPrefix(values, (x, y) -> x * y)`, the array contains

```
[1, 1 × 2, 1 × 2 × 3, 1 × 2 × 3  × 4, . . .]
```

Perhaps surprisingly, this computation can be parallelized. First, join neighboring elements, as indicated here:

```
[1, 1 × 2, 3, 3 × 4, 5, 5 × 6, 7, × 8]
```

The gray values are left alone. Clearly, one can make this computation in parallel in separate regions of the array. In the next step, update the indicated elements by multiplying them with elements that are one or two positions below:

```
[1, 1 × 2, 1 × 2 × 3, 1 × 2 × 3 × 4, 5, 5 × 6, 5 × 6 × 7, 5 × 6 × 7 × 8]
```

This, again, can be done in parallel. After log n steps, the process is complete. This is a win over the straightforward linear computation if sufficient processors are available. On special-purpose hardware, this algorithm is commonly used, and users of such hardware are quite ingenious in adapting it to a variety of problems.

10.6.8. Older Thread-Safe Collections

Ever since the initial release of Java, the `Vector` and `Hashtable` classes provided thread-safe implementations of a dynamic array and a hash table. These classes are now considered obsolete, having been replaced by the `ArrayList` and `HashMap` classes. Those classes are not thread-safe. Instead, a different mechanism is supplied in the collections library. Any collection class can be made thread-safe by means of a *synchronization wrapper*:

```
List<E> synchArrayList = Collections.synchronizedList(new ArrayList<>());
Map<K, V> synchHashMap = Collections.synchronizedMap(new HashMap<>());
```

The methods of the resulting collections use locking to provide thread-safe access.

You should make sure that no thread accesses the data structure through the original unsynchronized methods. The easiest way to ensure this is not to save any reference to the original object. Simply construct a collection and immediately pass it to the wrapper, as we did in our examples.

You still need to use "client-side" locking if you want to *iterate* over the collection while another thread has the opportunity to mutate it:

```
synchronized (synchHashMap)
{
  Iterator<K> iter = synchHashMap.keySet().iterator();
  while (iter.hasNext()) . . .;
}
```

You must use the same code if you use a "for each" loop because the loop uses an iterator. Note that the iterator actually fails with a `ConcurrentModificationException` if another thread mutates the collection while the iteration is in progress. The synchronization is still required so that the concurrent modification can be reliably detected.

You are usually better off using the collections defined in the `java.util.concurrent` package instead of the synchronization wrappers. In particular, the `ConcurrentHashMap` has

been carefully implemented so that multiple threads can access it without blocking each other, provided they access different buckets. One exception is an array list that is frequently mutated. In that case, a synchronized `ArrayList` can outperform a `CopyOnWriteArrayList`.

`java.util.Collections` **1.2**

- `static <E> Collection<E> synchronizedCollection(Collection<E> c)`
- `static <E> List synchronizedList(List<E> c)`
- `static <E> Set synchronizedSet(Set<E> c)`
- `static <E> SortedSet synchronizedSortedSet(SortedSet<E> c)`
- `static <K, V> Map<K, V> synchronizedMap(Map<K, V> c)`
- `static <K, V> SortedMap<K, V> synchronizedSortedMap(SortedMap<K, V> c)`

construct a view of the collection whose methods are synchronized.

10.7. Asynchronous Computations

So far, our approach to concurrent computation has been to break up a task, and then wait until all pieces have completed. Waiting works well with virtual threads, but with platform threads, a wait-free, or *asynchronous*, programming style provides higher throughput. The following sections are devoted to asynchronous computations.

10.7.1. Completable Futures

When you have a `Future` object, you need to call `get` to obtain the value, blocking until the value is available. The `CompletableFuture` class implements the `Future` interface, and it provides a second mechanism for obtaining the result. You register a *callback* that will be invoked (in some thread) with the result once it is available.

```
CompletableFuture<String> f = . . .;
f.thenAccept(s -> Process the result string s);
```

In this way, you can process the result without blocking once it is available.

There are a few API methods that return `CompletableFuture` objects. For example, you can fetch a web page asynchronously with the `HttpClient` class that you will encounter in **Chapter 4 of Volume II**:

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder(URI.create(urlString)).GET().build();
CompletableFuture<HttpResponse<String>> f = client.sendAsync(
  request, BodyHandlers.ofString());
```

It is nice if there is a method that produces a ready-made `CompletableFuture`, but most of the time, you need to make your own. To run a task asynchronously and obtain a `CompletableFuture`, you don't submit it directly to an executor service. Instead, you call the static method `CompletableFuture.supplyAsync`. Here is how to read the web page without the benefit of the `HttpClient` class:

```
public CompletableFuture<String> readPage(URI uri)
{
  return CompletableFuture.supplyAsync(() ->
    {
      try
      {
        return new String(uri.toURL().openStream().readAllBytes());
      }
      catch (IOException e)
      {
        throw new UncheckedIOException(e);
      }
    }, executor);
}
```

If you omit the executor, the task is run on a default executor (namely the executor returned by `ForkJoinPool.commonPool()`). You usually don't want to do that.



Caution: Note that the first parameter of the `supplyAsync` method is a `Supplier<T>`, not a `Callable<T>`. Both interfaces describe functions with no parameters and return type `T`, but a `Supplier` function cannot throw a checked exception. As you can see from the code above, that was not an inspired choice.

A `CompletableFuture` can complete in two ways: either with a result, or with an uncaught exception. In order to handle both cases, use the `whenComplete` method. The supplied function is called with the result (or `null` if none) and the exception (or `null` if none).

```
f.whenComplete((s, t) ->
  {
    if (t == null)
    {
      Process the result s;
    }
    else
    {
      Process the Throwable t;
    }
  });
```

The `CompletableFuture` is called completable because one can set a completion value. (In other concurrency libraries, such an entity is called a *promise*, and the act of completion is called "fulfilling the promise".) As a consumer of the `Future`, you don't care how the value is set. That is the job of the producer of the promise. The `supplyAsync` method produces a `CompletableFuture`, and it sets the completion value when the task has finished. Other producers can use a more complex approach. For example, this example produces a future where two tasks work simultaneously on completing it:

```
var f = new CompletableFuture<Integer>();
executor.execute(() ->
  {
    int n = workHard(arg);
    f.complete(n);
  });
executor.execute(() ->
  {
    int n = workSmart(arg);
    f.complete(n);
  });
executor.execute(() ->
  {
    try
    {
      Thread.sleep(timeout);
      f.completeExceptionally(new TimeoutException());
    }
    catch (InterruptedException e)
    {
```

```
        f.completeExceptionally(e);
      }
    });
```

It is safe to call `complete` or `completeExceptionally` on the same instance in multiple threads. If the future is already completed, these calls have no effect.



Caution: Unlike a plain `Future`, the computation of a `CompletableFuture` is not interrupted when you invoke its `cancel` method. Canceling simply completes the future exceptionally, with a `CancellationException`. In general, this makes sense since a `CompletableFuture` may not have a single thread that is responsible for its completion. However, this restriction also applies to `CompletableFuture` instances returned by methods such as `supplyAsync`, which could in principle be interrupted.

10.7.2. Composing Completable Futures

Nonblocking calls are implemented through callbacks. The programmer registers a callback for the action that should occur after a task completes. Of course, if the next action is also asynchronous, the next action after that is in a different callback. Even though the programmer thinks in terms of "first do step 1, then step 2, then step 3," the program logic can become dispersed in "callback hell." It gets even worse when one has to add error handling. Suppose step 2 is "the user logs in." You may need to repeat that step since the user can mistype the credentials. Trying to implement such a control flow in a set of callbacks, or to understand it once it has been implemented, can be quite challenging.

The `CompletableFuture` class addresses this problem by providing a mechanism for *composing* asynchronous tasks into a processing pipeline.

For example, suppose we want to extract all images from a web page. Let's say we have a method

```
public CompletableFuture<String> readPage(URI uri)
```

that yields the text of a web page when it becomes available. If the method

```
public List<URI> getImageLinks(String page)
```

yields the links of images in an HTML page, you can schedule it to be called when the page is available:

```
CompletableFuture<String> contents = readPage(uri);
CompletableFuture<List<URI>> imageLinks = contents.thenApply(this::getImageLinks);
```

The `thenApply` method doesn't block either. It returns another future. When the first future has completed, its result is fed to the `getImageLinks` method, and the return value of that method becomes the final result.

With completable futures, you just specify what you want to have done and in which order. It won't all happen right away, of course, but what is important is that all the code is in one place.

Conceptually, `CompletableFuture` is a simple API, but there are many variants of methods for composing completable futures. Let us first look at those that add an action to a single future (see **Table 10.3**). In the table, I use a shorthand notation for the ponderous functional interfaces, writing `T -> U` instead of `Function<? super T, ? extends U>`. These aren't actual Java types, of course.

You have already seen the `thenApply` method. Suppose `f` is a `T -> U` function. The call

```
CompletableFuture<U> future.thenApply(f);
```

returns a future that applies the function `f` to the result of `future` when it is available.

The `thenCompose` method, instead of taking a `T -> U` function, receives a function mapping `T` to `CompletableFuture<U>`. That sounds rather abstract, but it can be quite natural. It is a function that eventually yields a `U`.

The call

```
CompletableFuture<U> future.thenCompose(f);
```

works exactly like `thenApply`, applying `f` to the result of `future`. However, the result of `f` is not immediately available. When it arrives, the composition completes.

In the preceding section, you saw the `whenComplete` method for handling exceptions. There is also a `handle` method that requires a function processing the result or exception and computing a new result. In many cases, it is simpler to call the `exceptionally` method instead. That method computes a dummy value when an exception occurs:

```
CompletableFuture<List<URI>> imageLinks = readPage(uri)
  .exceptionally(ex -> "<html></html>")
  .thenApply(this::getImageLinks);
```

You can handle a timeout in the same way:

```
CompletableFuture<List<URI>> imageLinks = readPage(uri)
  .completeOnTimeout("<html></html>", 30, TimeUnit.SECONDS)
  .thenApply(this::getImageLinks);
```

Alternatively, you can throw an exception on timeout:

```
CompletableFuture<String> = readPage(uri).orTimeout(30, TimeUnit.SECONDS);
```

The methods in **Table 10.3** with `void` result are normally used at the end of a processing pipeline.

Table 10.3: Adding an Action to a `CompletableFuture<T>` Object

Method	Parameter	Description
`thenApply`	`T -> U`	Apply a function to the result.
`thenAccept`	`T -> void`	Like `thenApply`, but with `void` result.
`thenCompose`	`T -> CompletableFuture<U>`	Invoke the function on the result and execute the returned future.

Method	Parameter	Description
thenRun	Runnable	Execute the Runnable with void result.
handle	(T, Throwable) -> U	Process the result or error and yield a new result.
whenComplete	(T, Throwable) -> void	Like handle, but with void result.
exceptionally	Throwable -> U	Compute a result from the error.
exceptionallyCompose	Throwable -> CompletableFuture<U>	Invoke the function on the exception and execute the returned future.
completeOnTimeout	T, long, TimeUnit	Yield the given value as the result in case of timeout.
orTimeout	long, TimeUnit	Yield a TimeoutException in case of timeout.

Now let us turn to methods that combine multiple futures (see **Table 10.4**).

The first three methods run a `CompletableFuture<T>` and a `CompletableFuture<U>` action concurrently and combine the results.

The next three methods run two `CompletableFuture<T>` actions concurrently. As soon as one of them finishes, its result is passed on, and the other result is ignored.

Finally, the static `allOf` and `anyOf` methods shown in **Table 10.5** take a variable number of completable futures and yield a `CompletableFuture<Void>` that completes when all of them, or any one of them, completes. The `allOf` method does not yield a result. The `anyOf` method does *not* terminate the remaining tasks.



Caution: The `anyOf` may yield a future that completed exceptionally even if others completed successfully.

Table 10.4: Combining Another Future with a `CompletableFuture<T>`

Method	Parameters	Description
`thenCombine`	`CompletableFuture<U>`, `(T, U) -> V`	Execute both and combine the results with the given function.
thenAcceptBoth	CompletableFuture<U> , (T, U) -> void	Like thenCombine , but with void result.
`runAfterBoth`	`CompletableFuture<?>`, `Runnable`	Execute the runnable after both complete.
applyToEither	CompletableFuture<T> , T -> V	When a result is available from one or the other, pass it to the given function.
`acceptEither`	`CompletableFuture<T>`, `T -> void`	Like `applyToEither`, but with `void` result.
runAfterEither	CompletableFuture<?> , Runnable	Execute the runnable after one or the other completes.

Table 10.5: Combining Multiple Futures

Method	Parameter	Description
`static allOf`	`CompletableFuture<?>...`	Complete with `void` result after all given futures complete (some perhaps with an exception).
static anyOf	CompletableFuture<?>...	Complete after any of the given futures completes, yielding its result or exception.



Note: For each method shown, there are also two `Async` variants for specifying the executor of the "dependent completion" (that is, the action to be executed

upon completion of the future) executor. One of the variants uses a default executor (not recommended), and the other has an `Executor` parameter.

For a pipeline of actions, start off with an `Async` variant to set the desired executor, and then use the non-`Async` methods to continue with the same executor.



Note: Technically speaking, the methods in this section have parameters of type `CompletionStage`, not `CompletableFuture`. The `CompletionStage` interface describes how to compose asynchronous computations, whereas the `Future` interface focuses on the result of a computation. A `CompletableFuture` is both a `CompletionStage` and a `Future`.

Listing 10.10 shows a complete program that reads a web page, scans it for images, loads the images and saves them locally. Note how all asynchronous methods return a `CompletableFuture`. To kick off the computation, we use a little trick. Rather than calling the `readPage` method in a lambda expression passed to `supplyAsync`, a completed future provides the URI argument, which is then composed with `this::readPage`. That way, the pipeline has a very uniform appearance:

```
CompletableFuture.completedFuture(uri)
   .thenComposeAsync(this::readPage, executor)
   .thenApply(this::getImageLinks)
   .thenCompose(this::getImages)
   .thenAccept(this::saveImages);
```



Tip: This example shows the ideal form of an asynchronous pipeline. Each step is a function of one of the following three types:

- An asynchronous function `T -> CompletableFuture<U>` that yields a result in the future, passed to `thenCompose`
- A synchronous function `T -> U` that returns a result immediately, passed to `thenApply`
- A final consumer that returns no result, passed to `thenAccept`

The initial `completedFuture` provides the argument for the first function.

The first asynchronous step uses `composeAsync` to supply an executor.

Listing 10.10 completableFutures/CompletableFutureDemo.java

```
1  package completableFutures;
2
3  import java.awt.image.*;
4  import java.io.*;
5  import java.net.*;
6  import java.util.*;
7  import java.util.concurrent.*;
8  import java.util.regex.*;
9
10 import javax.imageio.*;
11
12 /**
13  * @version 1.02 2023-07-30
14  * @author Cay Horstmann
15  */
16 public class CompletableFutureDemo
17 {
18   private static final Pattern IMG_PATTERN = Pattern.compile(
19       "[<]\\s*[iI][mM][gG]\\s*[^>]*[sS][rR][cC]\\s*[=]\\s*['\"]([^'\"]*)['\"][^>]*[>]");
20   private ExecutorService executor = Executors.newCachedThreadPool();
21   private URI uriToProcess;
22
23   public CompletableFuture<String> readPage(URI uri)
24   {
25     return CompletableFuture.supplyAsync(() -> {
26       try
27       {
28         var contents = new String(uri.toURL().openStream().readAllBytes());
29         System.out.println("Read page from " + uri);
30         return contents;
31       }
32       catch (IOException e)
33       {
34         throw new UncheckedIOException(e);
35       }
36     }, executor);
37   }
38
39   public List<URI> getImageLinks(String webpage) // not blocking
40   {
41     var result = new ArrayList<URI>();
42     Matcher matcher = IMG_PATTERN.matcher(webpage);
43     while (matcher.find())
```

```
    {
      URI uri = URI.create(uriToProcess + "/" + matcher.group(1));
      result.add(uri);
    }
    System.out.println("Found links: " + result);
    return result;
  }

  public CompletableFuture<List<BufferedImage>> getImages(List<URI> uris)
  {
    return CompletableFuture.supplyAsync(() -> {
      try
      {
        var result = new ArrayList<BufferedImage>();
        for (URI uri : uris)
        {
          result.add(ImageIO.read(uri.toURL()));
          System.out.println("Loaded " + uri);
        }
        return result;
      }
      catch (IOException e)
      {
        throw new UncheckedIOException(e);
      }
    }, executor);
  }

  public void saveImages(List<BufferedImage> images)
  {
    System.out.println("Saving " + images.size() + " images");
    try
    {
      for (int i = 0; i < images.size(); i++)
      {
        String filename = "/tmp/image" + (i + 1) + ".png";
        ImageIO.write(images.get(i), "PNG", new File(filename));
      }
    }
    catch (IOException e)
    {
      throw new UncheckedIOException(e);
    }
    executor.shutdown();
  }
```

```
89
90    public CompletableFutureDemo(URI uri)
91    {
92       uriToProcess = uri;
93    }
94
95    public void run() throws IOException, InterruptedException
96    {
97       CompletableFuture.completedFuture(uriToProcess)
98             .thenComposeAsync(this::readPage, executor)
99             .thenApply(this::getImageLinks)
100            .thenCompose(this::getImages)
101            .thenAccept(this::saveImages);
102
103      // or use the HTTP client:
104      /*
105      HttpClient client = HttpClient.newBuilder().build();
106      HttpRequest request = HttpRequest.newBuilder(uriToProcess).GET().build();
107      client.sendAsync(request, BodyHandlers.ofString())
108         .thenApply(HttpResponse::body)
109         .thenApply(this::getImageLinks)
110         .thenCompose(this::getImages)
111         .thenAccept(this::saveImages);
112      */
113   }
114
115   public static void main(String[] args)
116         throws IOException, InterruptedException
117   {
118      new CompletableFutureDemo(URI.create("http://horstmann.com/index.html")).run();
119   }
120}
```

10.7.3. Long-Running Tasks in User-Interface Callbacks

One of the reasons to use threads is to make your programs more responsive. This is particularly important in an application with a user interface. With Swing, JavaFX, and Android, user-interface components are not thread-safe, and all user-interface actions run in a single thread. When your program needs to do something time-consuming, you cannot do the work in the user-interface thread, or the user interface will be frozen. Instead, fire up another worker thread.

For example, if you want to read a file when the user clicks a button, don't do this:

```
var open = new JButton("Open");
open.addActionListener(event ->
  { // BAD--long-running action is executed on UI thread
    var in = new Scanner(file);
    while (in.hasNextLine())
    {
      String line = in.nextLine();
      . . .
    }
  });
```

Instead, do the work in a separate thread.

```
open.addActionListener(event ->
  { // GOOD--long-running action in separate thread
    Runnable task = () ->
      {
        var in = new Scanner(file);
        while (in.hasNextLine())
        {
          String line = in.nextLine();
          . . .
        }
      };
    executor.execute(task);
  });
```

However, you cannot directly update the user interface from the worker thread that executes the long-running task. Since Swing, JavaFX, and Android are not thread-safe, you cannot manipulate user-interface elements from multiple threads, or they risk becoming corrupted. In fact, JavaFX and Android check for this, and throw an exception if you try to access the user interface from a thread other than the UI thread.

Therefore, you need to schedule any UI updates to happen on the UI thread. Each user-interface library provides some mechanism to schedule a `Runnable` for execution on the UI thread. For example, in Swing, you call

```
EventQueue.invokeLater(() -> label.setText(percentage + "% complete"));
```

It is tedious to implement user feedback in a worker thread, so each user-interface library provides some kind of helper class for managing the details, such as `SwingWorker` in Swing, `Task` in JavaFX, and `AsyncTask` in Android. You specify actions for the long-running task (which is run on a separate thread) as well as progress updates and the final disposition (which are run on the UI thread).

The program in **Listing 10.11** has commands for loading a text file and for canceling the file loading process. You should try the program with a long file, such as the full text of *The Count of Monte Cristo*, supplied in the `gutenberg` directory of the book's companion code. The file is loaded in a separate thread. While the file is being read, the Open menu item is disabled and the Cancel item is enabled (see **Figure 10.6**). After each line is read, a line counter in the status bar is updated. After the reading process is complete, the Open menu item is reenabled, the Cancel item is disabled, and the status line text is set to Done.

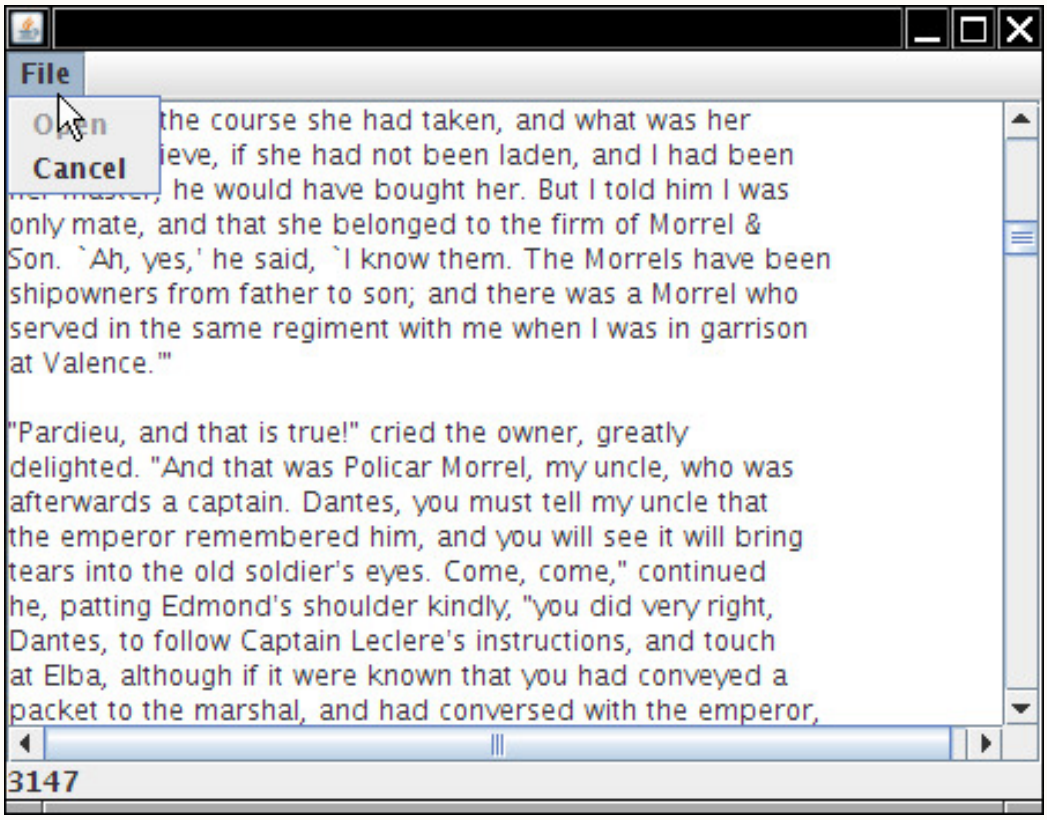


Figure 10.6: Loading a file in a separate thread

This example shows the typical UI activities of a background task:

- After each work unit, update the UI to show progress.
- After the work is finished, make a final change to the UI.

The `SwingWorker` class makes it easy to implement such a task. Override the `doInBackground` method to do the time-consuming work and occasionally call `publish` to communicate work progress. This method is executed in a worker thread. The `publish` method

causes a `process` method to execute in the event dispatch thread to deal with the progress data. When the work is complete, the `done` method is called in the event dispatch thread so that you can finish updating the UI.

Whenever you want to do some work in the worker thread, construct a new worker. (Each worker object is meant to be used only once.) Then call the `execute` method. You will typically call `execute` on the event dispatch thread, but that is not a requirement.

It is assumed that a worker produces a result of some kind; therefore, `SwingWorker<T, V>` implements `Future<T>`. This result can be obtained by the `get` method of the `Future` interface. Since the `get` method blocks until the result is available, you don't want to call it immediately after calling `execute`. It is a good idea to call it only when you know that the work has been completed. Typically, you call `get` from the `done` method. (There is no requirement to call `get`. Sometimes, processing the progress data is all you need.)

Both the intermediate progress data and the final result can have arbitrary types. The `SwingWorker` class has these types as type parameters. A `SwingWorker<T, V>` produces a result of type `T` and progress data of type `V`.

To cancel the work in progress, use the `cancel` method of the `Future` interface. When the work is canceled, the `get` method throws a `CancellationException`.

As already mentioned, the worker thread's call to `publish` will cause calls to `process` on the event dispatch thread. For efficiency, the results of several calls to `publish` may be batched up in a single call to `process`. The `process` method receives a `List<V>` containing all intermediate results.

Let us put this mechanism to work for reading in a text file. As it turns out, a `JTextArea` is quite slow. Appending lines from a long text file (such as all lines in *The Count of Monte Cristo*) takes considerable time.

To show the user that progress is being made, we want to display the number of lines read in a status line. Thus, the progress data consist of the current line number and the current line of text. We package these into a record:

```
private record ProgressData(int number, String line) {}
```

The final result is the text that has been read into a `StringBuilder`. Thus, we need a `SwingWorker<StringBuilder, ProgressData>`.

In the `doInBackground` method, we read a file, a line at a time. After each line, we call `publish` to publish the line number and the text of the current line.

```
@Override public StringBuilder doInBackground() throws IOException, InterruptedException
{
  int lineNumber = 0;
  var in = new Scanner(new FileInputStream(file));
  while (in.hasNextLine())
  {
    String line = in.nextLine();
    lineNumber++;
    text.append(line).append("\n");
    var data = new ProgressData(lineNumber, line);
    publish(data);
    Thread.sleep(1); // to test cancellation; no need to do this in your programs
  }
  return text;
}
```

We also sleep for a millisecond after every line so that you can test cancellation without getting stressed out, but you wouldn't want to slow down your own programs by sleeping. If you comment out this line, you will find that *The Count of Monte Cristo* loads quite quickly, with only a few batched user-interface updates.

In the `process` method, we ignore all line numbers but the last one, and we concatenate all lines for a single update of the text area.

```
@Override public void process(List<ProgressData> data)
{
  if (isCancelled()) return;
  var b = new StringBuilder();
  statusLine.setText("" + data.getLast().number());
  for (ProgressData d : data) b.append(d.line()).append("\n");
  textArea.append(b.toString());
}
```

In the `done` method, the text area is updated with the complete text, and the Cancel menu item is disabled.

Note how the worker is started in the event listener for the Open menu item.

This simple technique allows you to execute time-consuming tasks while keeping the user interface responsive.

Listing 10.11 swingWorker/SwingWorkerTest.java

```
1  package swingWorker;
2
3  import java.awt.*;
4  import java.io.*;
5  import java.util.*;
6  import java.util.List;
7  import java.util.concurrent.*;
8
9  import javax.swing.*;
10
11 /**
12  * This program demonstrates a worker thread that runs a potentially time-consuming task.
13  * @version 1.13 2023-10-11
14  * @author Cay Horstmann
15  */
16 public class SwingWorkerTest
17 {
18    public static void main(String[] args) throws Exception
19    {
20       EventQueue.invokeLater(() ->
21          {
22             var frame = new SwingWorkerFrame();
23             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24             frame.setVisible(true);
25          });
26    }
27 }
28
29 /**
30  * This frame has a text area to show the contents of a text file, a menu to open a file and
31  * cancel the opening process, and a status line to show the file loading progress.
32  */
33 class SwingWorkerFrame extends JFrame
34 {
35    private JFileChooser chooser;
36    private JTextArea textArea;
37    private JLabel statusLine;
```

```
38    private JMenuItem openItem;
39    private JMenuItem cancelItem;
40    private SwingWorker<StringBuilder, ProgressData> textReader;
41    public static final int TEXT_ROWS = 20;
42    public static final int TEXT_COLUMNS = 60;
43
44    public SwingWorkerFrame()
45    {
46       chooser = new JFileChooser();
47       chooser.setCurrentDirectory(new File("."));
48
49       textArea = new JTextArea(TEXT_ROWS, TEXT_COLUMNS);
50       add(new JScrollPane(textArea));
51
52       statusLine = new JLabel(" ");
53       add(statusLine, BorderLayout.SOUTH);
54
55       var menuBar = new JMenuBar();
56       setJMenuBar(menuBar);
57
58       var menu = new JMenu("File");
59       menuBar.add(menu);
60
61       openItem = new JMenuItem("Open");
62       menu.add(openItem);
63       openItem.addActionListener(event ->
64          {
65             // show file chooser dialog
66             int result = chooser.showOpenDialog(null);
67
68             // if file selected, set it as icon of the label
69             if (result == JFileChooser.APPROVE_OPTION)
70             {
71                textArea.setText("");
72                openItem.setEnabled(false);
73                textReader = new TextReader(chooser.getSelectedFile());
74                textReader.execute();
75                cancelItem.setEnabled(true);
76             }
77          });
78
79       cancelItem = new JMenuItem("Cancel");
80       menu.add(cancelItem);
81       cancelItem.setEnabled(false);
82       cancelItem.addActionListener(event -> textReader.cancel(true));
```

```
83      pack();
84    }
85
86    private record ProgressData(int number, String line) {}
87
88    private class TextReader extends SwingWorker<StringBuilder, ProgressData>
89    {
90      private File file;
91      private StringBuilder text = new StringBuilder();
92
93      public TextReader(File file)
94      {
95        this.file = file;
96      }
97
98      // the following method executes in the worker thread; it doesn't touch Swing components
99
100     public StringBuilder doInBackground() throws IOException, InterruptedException
101     {
102       int lineNumber = 0;
103       try (var in = new Scanner(new FileInputStream(file)))
104       {
105         while (in.hasNextLine())
106         {
107           String line = in.nextLine();
108           lineNumber++;
109           text.append(line).append("\n");
110           var data = new ProgressData(lineNumber, line);
111           publish(data);
112           Thread.sleep(1); // to test cancellation; no need to do this in your programs
113         }
114       }
115       return text;
116     }
117
118     // the following methods execute in the event dispatch thread
119
120     public void process(List<ProgressData> data)
121     {
122       if (isCancelled()) return;
123       var builder = new StringBuilder();
124       statusLine.setText("" + data.getLast().number());
125       for (ProgressData d : data) builder.append(d.line()).append("\n");
126       textArea.append(builder.toString());
127     }
```

```
128
129     public void done()
130     {
131       try
132       {
133         StringBuilder result = get();
134         textArea.setText(result.toString());
135         statusLine.setText("Done");
136       }
137       catch (InterruptedException ex)
138       {
139       }
140       catch (CancellationException ex)
141       {
142         textArea.setText("");
143         statusLine.setText("Cancelled");
144       }
145       catch (ExecutionException ex)
146       {
147         statusLine.setText("" + ex.getCause());
148       }
149
150       cancelItem.setEnabled(false);
151       openItem.setEnabled(true);
152     }
153   }
154 }
```

javax.swing.SwingWorker<T, V> **6**

- `abstract T doInBackground()`
  is the method to override to carry out the background task and to return the result of the work.
- `void process(List<V> data)`
  is the method to override to process intermediate progress data in the event dispatch thread.
- `void publish(V... data)`
  forwards intermediate progress data to the event dispatch thread. Call this method from `doInBackground`.
- `void execute()`
  schedules this worker for execution on a worker thread.

- `SwingWorker.StateValue getState()`
  gets the state of this worker—one of `PENDING`, `STARTED`, or `DONE`.

10.8. Processes

Up to now, you have seen how to execute Java code in separate threads within the same program. Sometimes, you need to execute another program. For this, use the `ProcessBuilder` and `Process` classes. The `Process` class executes a command in a separate operating system process and lets you interact with its standard input, output, and error streams. The `ProcessBuilder` class lets you configure a `Process` object.



Note: The `ProcessBuilder` class is a more flexible replacement for the `Runtime.exec` calls.

10.8.1. Building a Process

Start by specifying the command that you want to execute. You can supply a `List<String>` or simply the strings that make up the command.

```
var builder = new ProcessBuilder("gcc", "myapp.c");
```



Caution: The first string must be an executable command, not a shell builtin. For example, to run the `dir` command in Windows, you need to build a process with strings `"cmd.exe"`, `"/C"`, and `"dir"`.

Each process has a *working directory*, which is used to resolve relative directory names. By default, a process has the same working directory as the virtual machine, which is typically the directory from which you launched the `java` program. You can change it with the `directory` method:

```
builder = builder.directory(path.toFile());
```



Note: Each of the methods for configuring a `ProcessBuilder` returns itself, so that you can chain commands. Ultimately, you will call

```
Process p = new ProcessBuilder(command).directory(file)....start();
```

Next, you will want to specify what should happen to the standard input, output, and error streams of the process. By default, each of them is a pipe that you can access with

```
OutputStream processIn = p.getOutputStream();
InputStream processOut = p.getInputStream();
InputStream processErr = p.getErrorStream();
```

or, for text input and output

```
BufferedWriter processIn = p.outputWriter();
BufferedReader processOut = p.inputReader();
BufferedReader processErr = p.errorReader();
```

Note that the input stream of the process is an output stream in the JVM! You write to that stream, and whatever you write becomes the input of the process. Conversely, you read what the process writes to the output and error streams. For you, they are input streams. The same holds for readers and writers.

You can specify that the input, output, and error streams of the new process should be the same as the JVM. If the user runs the JVM in a console, any user input is forwarded to the process, and the process output shows up in the console. Call

```
builder.inheritIO();
```

to make this setting for all three streams. If you only want to inherit some of the streams, pass the value

```
ProcessBuilder.Redirect.INHERIT
```

to the `redirectInput`, `redirectOutput`, or `redirectError` methods. For example,

```
builder.redirectOutput(ProcessBuilder.Redirect.INHERIT);
```

You can redirect the process streams to files by supplying `File` objects:

```
builder.redirectInput(inputFile)
  .redirectOutput(outputFile)
  .redirectError(errorFile)
```

;
The files for output and error are created or truncated when the process starts. To append to existing files, use

```
builder.redirectOutput(ProcessBuilder.Redirect.appendTo(outputFile));
```

It is often useful to merge the output and error streams, so you can see the outputs and error messages in the sequence in which the process generates them. Call

```
builder.redirectErrorStream(true);
```

to activate the merging. If you do that, you can no longer call `redirectError` on the `ProcessBuilder` or `getErrorStream` on the `Process`.

You may also want to modify the environment variables of the process. Here, the builder chain syntax breaks down. You need to get the builder's environment (which is initialized by the environment variables of the process running the JVM), then put or remove entries.

```
Map<String, String> env = builder.environment();
env.put("LANG", "fr_FR");
env.remove("JAVA_HOME");
Process p = builder.start();
```

If you want to pipe the output of one process into the input of another (as with the `|` operator in a shell), use the `startPipeline` method. Pass a list of process builders and read the result from the last process. Here is an example, enumerating the unique extensions in a directory tree:

```
List<Process> processes = ProcessBuilder.startPipeline(List.of(
  new ProcessBuilder("find", "/opt/jdk-21"),
  new ProcessBuilder("grep", "-o", "\\.[^./]*$"),
  new ProcessBuilder("sort"),
  new ProcessBuilder("uniq")
```

```
));
Process last = processes.getLast();
var result = new String(last.getInputStream().readAllBytes());
```

Of course, this particular task would be more efficiently solved by making the directory walk in Java instead of running four processes. **Chapter 2 of Volume II** will show you how to do that.

10.8.2. Running a Process

After you have configured the builder, invoke its `start` method to start the process. If you configured the input, output, and error streams as pipes, you can now write to the input stream and read the output and error streams. For example,

```
Process process = new ProcessBuilder("/bin/ls", "-l")
  .directory(Path.of("/tmp").toFile())
  .start();
try (var in = new Scanner(process.getInputStream()))
{
  while (in.hasNextLine())
    System.out.println(in.nextLine());
}
```



Caution: There is limited buffer space for the process streams. You should not flood the input, and you should read the output promptly. If there is a lot of input and output, you may need to produce and consume it in separate threads.

To wait for the process to finish, call

```
int result = process.waitFor();
```

or, if you don't want to wait indefinitely,

```
long delay = . . .;
if (process.waitFor(delay, TimeUnit.SECONDS))
{
  int result = process.exitValue();
  . . .
}
```

```
else
{
  process.destroyForcibly();
}
```

The first call to `waitFor` returns the exit value of the process (by convention, `0` for success or a nonzero error code). The second call returns `true` if the process didn't time out. Then you need to retrieve the exit value by calling the `exitValue` method.

Instead of waiting for the process to finish, you can just leave it running and occasionally call `isAlive` to see whether it is still alive. To kill the process, call `destroy` or `destroyForcibly`. The difference between these calls is platform-dependent. On UNIX, the former terminates the process with `SIGTERM`, the latter with `SIGKILL`. (The `supportsNormalTermination` method returns `true` if the `destroy` method can terminate the process normally.)

Finally, you can receive an asynchronous notification when the process has completed. The call `process.onExit()` yields a `CompletableFuture<Process>` that you can use to schedule any action.

```
process.onExit().thenAccept(
  p -> System.out.println("Exit value: " + p.exitValue()));
```



Note: Since Java 21, the logger with name `java.lang.ProcessBuilder` logs processes executed by `Runtime.exec` and `ProcessBuilder` at the logging levels `DEBUG` and `TRACE`. The command arguments are only included at the `TRACE` level. Both levels log the process id, directory, command, and stack trace.

10.8.3. Process Handles

To get more information about a process that your program started, or any other process that is currently running on your machine, use the `ProcessHandle` interface. You can obtain a `ProcessHandle` in four ways:

1. Given a `Process` object `p`, `p.toHandle()` yields its `ProcessHandle`.
2. Given a `long` operating system process ID, `ProcessHandle.of(id)` yields the handle of that process.
3. `ProcessHandle.current()` is the handle of the process that runs this Java virtual machine.

4. `ProcessHandle.allProcesses()` yields a `Stream<ProcessHandle>` of all operating system processes that are visible to the current process.

Given a process handle, you can get its process ID, its parent process, its children, and its descendants:

```
long pid = handle.pid();
Optional<ProcessHandle> parent = handle.parent();
Stream<ProcessHandle> children = handle.children();
Stream<ProcessHandle> descendants = handle.descendants();
```



Note: The `Stream<ProcessHandle>` instances that are returned by the `allProcesses`, `children`, and `descendants` methods are just snapshots in time. Any of the processes in the stream might be terminated by the time you get around to seeing them, and other processes may have started that are not in the stream.

The `info` method yields a `ProcessHandle.Info` object with methods for obtaining information about the process.

```
Optional<String[]> arguments()
Optional<String> command()
Optional<String> commandLine()
Optional<Instant> startInstant()
Optional<Duration> totalCpuDuration()
Optional<String> user()
```

All of these methods return `Optional` values since it is possible that a particular operating system may not be able to report the information.

For monitoring or forcing process termination, the `ProcessHandle` interface has the same `isAlive`, `supportsNormalTermination`, `destroy`, `destroyForcibly`, and `onExit` methods as the `Process` class. However, there is no equivalent to the `waitFor` method.

Listing 10.12 shows how to start a process and read its output, and how to list all Java processes.

Listing 10.12 process/ProcessDemo.java

```
1  package process;
2
3  import java.io.IOException;
4  import java.nio.file.Path;
5  import java.util.Scanner;
6
7  /**
8   * This program demonstrates running a process and reading its output, and listing all Java
9   * processes
10  * @version 1.1 2023-10-11
11  * @author Cay Horstmann
12  */
13 public class ProcessDemo
14 {
15    public static void main(String[] args) throws IOException, InterruptedException
16    {
17       Process p = new ProcessBuilder("/bin/ls", "-l")
18          .directory(Path.of("/tmp").toFile())
19          .start();
20       try (var in = new Scanner(p.getInputStream()))
21       {
22          while (in.hasNextLine())
23             System.out.println(in.nextLine());
24       }
25       System.out.println("pid: " + p.toHandle().pid());
26       int result = p.waitFor();
27       System.out.println("Exit value: " + result);
28
29       ProcessHandle.allProcesses()
30          .map(ProcessHandle::info)
31          .filter(info -> info.command().filter(s -> s.contains("java")).isPresent())
32          .forEach(info -> info.commandLine().ifPresent(System.out::println));
33    }
34 }
```

java.lang.ProcessBuilder **5.0**

- `ProcessBuilder(String... command)`
- `ProcessBuilder(List<String> command)`

  construct a process builder with the given command and arguments.

- `ProcessBuilder directory(File directory)`
  sets the working directory for the process.
- `ProcessBuilder inheritIO()` 9
  makes the process use the standard input, output, and error of the virtual machine.
- `ProcessBuilder redirectErrorStream(boolean redirectErrorStream)`
  If `redirectErrorStream` is `true`, the standard error of the process is merged into the standard output.
- `ProcessBuilder redirectInput(File file)` 7
- `ProcessBuilder redirectOutput(File file)` 7
- `ProcessBuilder redirectError(File file)` 7
  redirect the standard input, output, or error of the process to the given file.
- `ProcessBuilder redirectInput(ProcessBuilder.Redirect source)` 7
- `ProcessBuilder redirectOutput(ProcessBuilder.Redirect destination)` 7
- `ProcessBuilder redirectError(ProcessBuilder.Redirect destination)` 7
  redirect the standard input, output, or error of the process, where `destination` is one of:
  - `Redirect.PIPE` —the default behavior, access via the `Process` object
  - `Redirect.INHERIT` —the stream from the virtual machine
  - `Redirect.DISCARD`
  - `Redirect.from(file)`
  - `Redirect.to(file)`
  - `Redirect.appendTo(file)`
- `Map<String, String> environment()`
  yields a mutable map for setting environment variables for the process.
- `Process start()`
  starts the process and yields its `Process` object.
- `static List<Process> startPipeline(List<ProcessBuilder> builders)` 9
  starts a pipeline of processes, connecting the standard output of each process to the standard input of the next one.

`java.lang.Process` 1.0

- `abstract OutputStream getOutputStream()`
  gets a stream for writing to the input stream of the process.
- `abstract InputStream getInputStream()`
- `abstract InputStream getErrorStream()`
  get an input stream for reading the output or error stream of the process.

- `abstract int waitFor()`
  waits for the process to finish and yields the exit value.
- `boolean waitFor(long timeout, TimeUnit unit)` **8**
  waits for the process to finish, but no longer than the given timeout. Returns `true` if the process exited.
- `abstract int exitValue()`
  returns the exit value of the process. By convention, a nonzero exit value indicates an error.
- `boolean isAlive()` **8**
  checks whether this process is still alive.
- `abstract void destroy()`
- `Process destroyForcibly()` **8**
  terminate this process, either normally or forcefully.
- `boolean supportsNormalTermination()` **9**
  checks whether this process can be terminated normally or must be destroyed forcefully.
- `ProcessHandle toHandle()` **9**
  yields the `ProcessHandle` describing this process.
- `CompletableFuture<Process> onExit()` **9**
  yields a `CompletableFuture` that is executed when this process exits.

java.lang.ProcessHandle **9**

- `static Optional<ProcessHandle> of(long pid)`
- `static Stream<ProcessHandle> allProcesses()`
- `static ProcessHandle current()`
  yield the process handle(s) of the process with the given PID, of all processes, or the process of the virtual machine.
- `Stream<ProcessHandle> children()`
- `Stream<ProcessHandle> descendants()`
  yield the process handles of the children or descendants of this process.
- `long pid()`
  yields the PID of this process.
- `ProcessHandle.Info info()`
  yields detail information about this process.

java.lang.ProcessHandle.Info 9

- `Optional<String[]> arguments()`
- `Optional<String> command()`
- `Optional<String> commandLine()`
- `Optional<Instant> startInstant()`
- `Optional<Duration> totalCpuDuration()`
- `Optional<String> user()`

yield the given detail information if available.

Previous chapter
< 9. Collections

Next chapter
11. Annotations >